

绿色限行下异构车队调度的递进优化模型： GRASP-VND、政策感知 ALNS 与滚动时域响应

摘要

城市绿色物流配送调度需要同时兼顾运营成本、客户时间窗、异构车辆容量、时变交通状态与绿色低碳政策。针对题目给定的 98 个客户点和 2169 个订单，本文构建“能送完-送得省-送得合规-送得稳”的递进式建模框架，依次求解静态调度、绿色限行政策下调度和动态事件响应调度三类问题。

针对问题一，首先将订单聚合为客户需求并核验绿色配送区客户，在考虑重量-体积双容量、软时间窗、时变速度、油电能耗和碳排放成本的基础上，建立异构车队车辆路径优化模型。求解上先用最近邻贪心构造可行基准解，再采用 GRASP 生成多起点初始解，并结合 2-opt、relocate、swap、车型再分配和发车时刻搜索形成 GRASP-VND 算法。最终静态方案使用 121 辆车，总里程为 14574.78 km，总碳排放为 10211.80 kg，总成本为 86122.34 元；相较 GRASP+2-opt 基准解，成本降低 1985.77 元。

针对问题二，在问题一方案基础上加入 8:00-16:00 燃油车不得服务绿色配送区客户的政策约束，设计政策约束感知 ALNS 算法，通过绿色区冲突移除、延后发车、路线重排序和车型再分配进行局部修复。结果表明，燃油车 16:00 前服务绿色区客户的违规次数由 16 次降为 0；政策实施后总成本为 87957.53 元，较问题一增加 2.13%，总碳排放为 10481.14 kg，增加 2.64%。

针对问题三，构建事件触发的滚动时域动态 ALNS 模型，冻结已完成和在途任务，仅对受影响路线局部重优化。对订单取消、新增绿色区订单和时间窗收紧三个事件仿真后，最终总成本为 87669.94 元，总里程为 14497.85 km，总碳排放为 10401.27 kg，限行违规次数仍为 0，调整路线比例仅为 3.31%。结果说明本文方法可在保持政策合规的同时实现低扰动、可执行的动态调度。

关键字：绿色物流；异构车队；软时间窗；GRASP-VND；ALNS；滚动时域调度

目录

一、	问题重述	3
1.1	问题背景	3
1.2	待解决问题	3
二、	问题分析	3
2.1	问题一分析	3
2.2	问题二分析	4
2.3	问题三分析	4
2.4	总体技术路线	4
三、	模型假设	4
四、	符号说明	5
五、	数据预处理	5
5.1	订单聚合与需求重构	5
5.2	绿色配送区识别	6
5.3	车辆参数	7
5.4	时变速度与能耗处理	7
六、	问题一：静态环境下的车辆调度模型	7
6.1	模型建立	7
6.1.1	决策变量	7
6.1.2	目标函数	8
6.1.3	约束条件	8
6.2	求解算法	9
6.2.1	贪心基准解	9
6.2.2	GRASP-VND 静态优化	9
6.3	问题一求解结果	9
6.3.1	总体成本	9
6.3.2	车型使用结果	10
6.3.3	算法递进效果	11
6.3.4	路线和到达时间	12
七、	问题二：环保政策影响下的车辆调度策略	13
7.1	绿色配送区限行模型	13
7.2	政策感知 ALNS 算法	14
7.2.1	破坏算子	14
7.2.2	修复算子	14
7.3	问题二结果分析	14
7.3.1	政策冲突消除效果	14
7.3.2	成本与碳排放变化	15
7.3.3	ALNS 修复过程与绿色区服务时刻校验	17
7.3.4	问题二部分路径示例	18
八、	问题三：动态事件下的实时车辆调度策略	18
8.1	滚动时域动态调度模型	18
8.2	滚动时域 ALNS 求解策略	19
8.3	动态事件设置与响应结果	20
8.4	事件 E2：新增绿色区订单插入	21
8.5	事件 E3：时间窗收紧修复	22
8.6	问题三最终结果	23
九、	模型评价与进一步改进	24
9.1	模型优点	24
9.2	模型不足	25
9.3	政策含义与改进方向	25
附录 A	支撑材料文件列表	26
附录 B	源程序代码	27

一、 问题重述

1.1 问题背景

随着电子商务和城市即时配送快速发展，物流企业每日需要在有限车辆资源下完成大量客户订单配送。与此同时，城市低碳政策逐步加强，燃油车在中心城区或绿色配送区内受到通行时段限制。企业调度方案不能只追求最短距离，还必须同时考虑车辆启动成本、油电能耗成本、碳排放成本、客户时间窗满意度和政策合规性。

本题给定 98 个客户点、距离矩阵、客户坐标、时间窗、订单需求以及五类车辆参数。配送车辆包括三类燃油车和两类新能源车。客户时间窗为软约束，早到产生等待成本，晚到产生惩罚成本；车速随交通状态分段变化，油耗或电耗与速度呈 U 型关系，且满载车辆能耗高于空载车辆。绿色配送区以市中心 (0,0) 为圆心、半径 10 km，问题二要求 8:00–16:00 禁止燃油车进入该区域。

1.2 待解决问题

问题 1：在无绿色限行政策条件下，建立静态车辆调度模型，给出车辆使用方案、行驶路径、到达时间和成本构成。

问题 2：在绿色配送区限行政策下重新规划路径，分析政策对总成本、车辆使用结构和碳排放总量的影响。

问题 3：面对订单取消、新增订单、配送地址变更或时间窗调整等突发事件，设计实时响应的动态调度策略，并给出典型事件下的路径调整结果。

二、 问题分析

2.1 问题一分析

问题一属于带异构车队和软时间窗的车辆路径问题。与普通 VRP 相比，本题有四个关键难点：

- (1) **异构车辆：**车辆的载重、容积、动力类型和数量不同，路线与车型之间存在匹配问题。
- (2) **双容量约束：**客户需求同时包含重量和体积，且部分客户需求超过单车容量，需要拆分配送。
- (3) **时变速度：**行驶时间和能耗受出发时刻影响，路线顺序会改变车辆进入不同时段交通状态。
- (4) **软时间窗：**客户早到等待和晚到惩罚均计入目标函数，因此不能只优化距离。

因此，问题一宜先构造可行方案，再通过多邻域搜索优化路线顺序、客户分配和车型匹配。

2.2 问题二分析

问题二在问题一基础上加入绿色配送区限行政策。由于附件只提供客户坐标和点间道路距离，而没有完整道路轨迹，本文将绿色限行解释为节点服务约束：燃油车在 8:00–16:00 不得服务绿色配送区客户。该解释避免了在缺少道路轨迹时主观判断车辆是否穿越绿色区，同时能准确刻画客户服务时刻的政策合规性。

绿色限行会使问题一中部分燃油车路线失效。若完全从零重求，会破坏原有调度方案稳定性；若只做单点交换，又难以同时兼顾成本和时间窗。因此，本文以问题一解为基准，采用政策感知 ALNS 进行局部破坏与修复。

2.3 问题三分析

问题三进一步从静态计划扩展到配送过程中的动态事件响应。若每次突发事件都重新求解全局方案，会导致大量车辆和司机任务被改变，实际执行成本较高；若完全不调整，又可能无法处理新增订单或收紧后的时间窗。因此，问题三适合采用滚动时域思想：在事件发生时冻结已完成任务和在途任务，仅抽取未执行且受影响的客户、路线和候选车辆进行局部重优化。该策略的评价标准不仅包括成本和碳排放，还应包括调整路线比例、新增/撤销路线数以及最终政策违规次数，从而体现动态方案的稳定性和可执行性。

2.4 总体技术路线

本文采用递进式建模与算法优化路线：

数据清洗 → 贪心可行解 → GRASP-VND 静态优化
→ 政策感知 ALNS 修复 → 滚动时域 ALNS 动态响应。

其中，贪心算法用于建立可行基准，GRASP-VND 用于解决问题一，政策感知 ALNS 用于在问题一解基础上处理问题二的限行冲突。

三、 模型假设

- 假设 1： 配送中心编号为 0，客户编号为 $1, 2, \dots, 98$ ；所有车辆从配送中心出发并最终返回配送中心。
- 假设 2： 订单可在客户层面聚合，部分大需求客户允许由多辆车拆分配送，但同一客户的总配送重量和总体积必须被完全满足。
- 假设 3： 单辆车在一条路线上的总配送重量和总体积分别不超过该车型载重和容积上限。
- 假设 4： 客户时间窗为软约束，早到等待和晚到惩罚均计入成本；配送中心等待不计入客户等待成本。
- 假设 5： 车辆服务单个客户的服务时间固定为 20 分钟。
- 假设 6： 在同一交通状态内，车速用题目给定均值和方差刻画；跨时段行驶时按时间段切分后累计行驶时间与能耗。

假设 7：问题二中绿色配送区限行按客户服务节点进行判断，即燃油车不得在 8:00–16:00 服务绿色区客户。

假设 8：问题三中，突发事件发生前已经完成的服务任务不可被回滚，在途车辆继续执行已确认任务；仅未出发路线或受影响路线允许被局部调整。

四、符号说明

表 1 主要符号说明

符号	含义	单位
0	配送中心编号	—
C	客户集合, $C = \{1, 2, \dots, n\}$	—
V	节点集合, $V = \{0\} \cup C$	—
H	车型集合	—
K_h	车型 h 对应车辆集合	—
Q_h	车型 h 最大载重	kg
B_h	车型 h 最大容积	m ³
N_h	车型 h 可用车辆数量	辆
w_i	客户 i 的总需求重量	kg
b_i	客户 i 的总需求体积	m ³
d_{ij}	节点 i 到节点 j 的道路距离	km
$[e_i, l_i]$	客户 i 的软时间窗	h
s_i	客户 i 服务时间, 本题取 1/3 h	h
a_i^k	车辆 k 到达客户 i 的时刻	h
x_{ij}^k	车辆 k 是否从 i 行驶至 j	—
z_i^k	车辆 k 是否服务客户 i	—
q_i^k, r_i^k	车辆 k 向客户 i 配送的重量和体积	kg, m ³
θ_{ij}^k	车辆 k 在弧 (i, j) 上的剩余载重率	—
G	绿色配送区客户集合	—
g_i	客户 i 是否属于绿色配送区	—
f_k	车辆 k 是否为燃油车	—

五、数据预处理

5.1 订单聚合与需求重构

设 Ω_i 为客户 i 对应订单集合，将订单级数据按客户编号聚合，得到客户级需求：

$$w_i = \sum_{r \in \Omega_i} w_{ir}, \quad b_i = \sum_{r \in \Omega_i} b_{ir}. \quad (1)$$

经清洗后，数据包含 1 个配送中心和 98 个客户点，其中 88 个客户点存在正需求。总需求重量为 286140.35 kg，总体积为 781.54 m³。由于客户需求差异较大，部分客户的单点需求超过任一车辆最大容量，故本文采用可拆分配送处理。

5.2 绿色配送区识别

题目中的配送中心坐标为 (20, 20)，绿色配送区圆心为市中心 (0, 0)，二者不能混用。根据客户坐标 (X_i, Y_i) ，定义客户到市中心距离为

$$r_i = \sqrt{X_i^2 + Y_i^2}. \quad (2)$$

绿色配送区客户集合为

$$G = \{i \in C \mid r_i \leq 10\}. \quad (3)$$

经坐标核验，共有 15 个客户点位于绿色配送区，其中 12 个客户存在实际需求。图1 展示了客户空间分布、配送中心、市中心和绿色配送区边界。

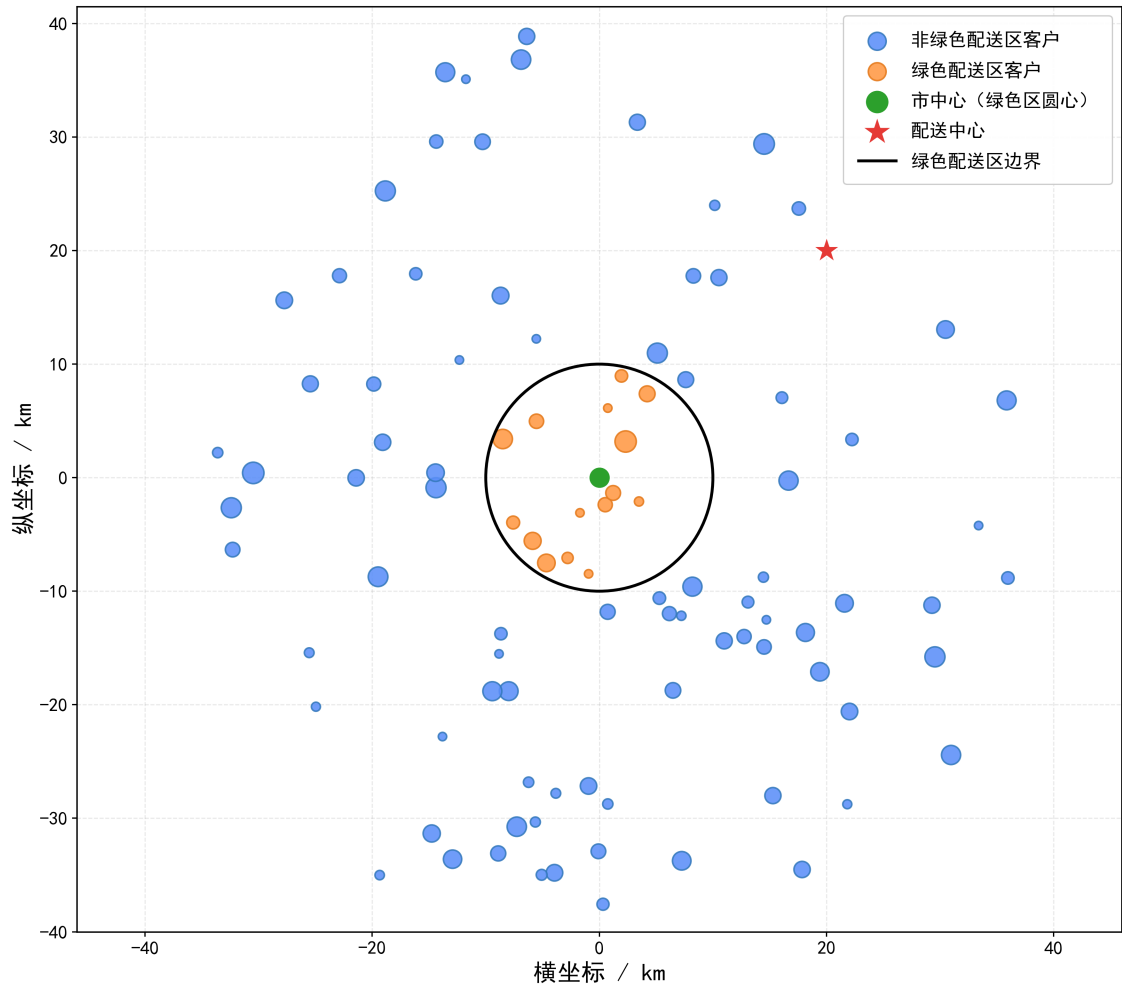


图 1 客户空间分布与绿色配送区

5.3 车辆参数

表 2 车辆类型与容量参数

车辆类型	动力类型	最大载重/kg	最大容积/m ³	可用数量/辆
燃油大车	燃油	3000	13.5	60
燃油中车	燃油	1500	10.8	50
燃油小车	燃油	1250	6.5	50
新能源大车	电动	3000	15.0	10
新能源小车	电动	1250	8.5	15

5.4 时变速度与能耗处理

以 8:00 为时间原点, 用 t 表示从 8:00 起算的小时数。交通速度按题目给定时段划分为拥堵、顺畅和一般三个状态。对于速度二次项, 本文使用

$$E(v^2) = \mu^2 + \sigma^2 \quad (4)$$

将速度随机性纳入油电耗计算。燃油车百公里油耗和新能源车百公里电耗分别为

$$\text{FPK} = 0.0025(\mu^2 + \sigma^2) - 0.2554\mu + 31.75, \quad (5)$$

$$\text{EPK} = 0.0014(\mu^2 + \sigma^2) - 0.12\mu + 36.19. \quad (6)$$

考虑满载能耗高于空载, 车辆 k 在弧 (i, j) 上的载重修正系数设为

$$\rho_{ij}^k = \begin{cases} 1 + 0.40\theta_{ij}^k, & k \in K_F, \\ 1 + 0.35\theta_{ij}^k, & k \in K_E, \end{cases} \quad (7)$$

其中 K_F 和 K_E 分别表示燃油车集合与新能源车集合。

六、 问题一：静态环境下的车辆调度模型

6.1 模型建立

6.1.1 决策变量

定义二元变量

$$x_{ij}^k = \begin{cases} 1, & \text{车辆 } k \text{ 从节点 } i \text{ 行驶到节点 } j, \\ 0, & \text{否则,} \end{cases} \quad z_i^k = \begin{cases} 1, & \text{车辆 } k \text{ 服务客户 } i, \\ 0, & \text{否则.} \end{cases} \quad (8)$$

令 q_i^k 和 r_i^k 分别为车辆 k 对客户 i 配送的重量和体积。

6.1.2 目标函数

问题一以总配送成本最低为目标：

$$\min C_{\text{total}} = C_{\text{start}} + C_{\text{energy}} + C_{\text{carbon}} + C_{\text{wait}} + C_{\text{late}}. \quad (9)$$

车辆启动成本为

$$C_{\text{start}} = 400 \sum_{k \in K} y_k. \quad (10)$$

软时间窗成本为

$$C_{\text{wait}} = 20 \sum_{k \in K} \sum_{i \in C} \max(e_i - a_i^k, 0) z_i^k, \quad (11)$$

$$C_{\text{late}} = 50 \sum_{k \in K} \sum_{i \in C} \max(a_i^k - l_i, 0) z_i^k. \quad (12)$$

燃油价格取 7.61 元/L，电价取 1.64 元/kW·h，碳排放成本取 0.65 元/kg，燃油和电力碳排放系数分别为 2.547 kg/L 和 0.501 kg/kW·h。

6.1.3 约束条件

客户需求完整配送约束为

$$\sum_{k \in K} q_i^k = w_i, \quad \sum_{k \in K} r_i^k = b_i, \quad \forall i \in C. \quad (13)$$

车辆容量约束为

$$\sum_{i \in C} q_i^k \leq Q_k, \quad \sum_{i \in C} r_i^k \leq B_k, \quad \forall k \in K. \quad (14)$$

配送量与访问变量满足

$$0 \leq q_i^k \leq Q_k z_i^k, \quad 0 \leq r_i^k \leq B_k z_i^k. \quad (15)$$

车辆从配送中心出发并返回配送中心：

$$\sum_{j \in C} x_{0j}^k = y_k, \quad \sum_{i \in C} x_{i0}^k = y_k. \quad (16)$$

车辆流量守恒约束为

$$\sum_{i \in V, i \neq h} x_{ih}^k = \sum_{j \in V, j \neq h} x_{hj}^k = z_h^k, \quad \forall h \in C, \forall k \in K. \quad (17)$$

时间递推约束为

$$a_j^k \geq a_i^k + s_i + \tau_{ij}(a_i^k) - M(1 - x_{ij}^k), \quad \forall i, j \in V, i \neq j. \quad (18)$$

车辆数量不能超过题目给定上限：

$$\sum_{k \in K_h} y_k \leq N_h, \quad \forall h \in H. \quad (19)$$

6.2 求解算法

6.2.1 贪心基准解

首先构造最近邻贪心基准解。算法从配送中心出发，在满足容量和体积约束的候选客户中选择综合插入代价最小的客户；当当前车辆无法继续插入客户时，该车返回配送中心并启用下一辆车。该方法计算速度快，但容易被局部最近客户选择锁定，因此只作为后续优化的可行基准。

6.2.2 GRASP-VND 静态优化

在贪心解基础上，本文采用 GRASP-VND 求解问题一。GRASP 通过随机限制候选表生成多组初始解，提升搜索多样性；VND 依次采用多个邻域结构改进路径。本文使用的邻域包括：

- (1) **2-opt 路线内优化**：反转同一路线中的局部访问顺序，减少绕行距离。
- (2) **relocate 单点重插入**：将某条路线中的客户移动到另一条路线的合适位置，改善容量利用率和时间窗成本。
- (3) **swap 双点交换**：交换两条路线中的客户访问点，缓解单点移动难以跳出的局部结构。
- (4) **车型全局再分配**：在路线客户集合基本固定后，重新匹配路线与车型，降低油电成本和碳成本。
- (5) **路线顺序精确优化**：对停靠点较少的路线枚举访问顺序，选择成本最低的排列。

此外，程序对每条路线以 5 分钟为步长搜索发车时刻，允许车辆在配送中心等待后出发，以平衡时变速度和客户时间窗成本。

问题一 GRASP-VND 求解流程

- Step1: 读入客户节点、距离矩阵、时间窗和车辆参数。
- Step2: 用贪心算法生成确定性基准解。
- Step3: 通过 GRASP 生成多组随机初始解，并执行 2-opt 路线内优化。
- Step4: 进入 VND 阶段，依次执行 relocate、swap、车型再分配和路线顺序精确优化。
- Step5: 对每条路线搜索最佳发车时刻，重新计算成本、到达时刻和碳排放。
- Step6: 输出路线汇总、到达时间明细、成本汇总和可视化结果。

6.3 问题一求解结果

6.3.1 总体成本

问题一最终方案使用 121 辆车，完成全部客户需求配送。由于 10 辆新能源大车、60 辆燃油大车和 50 辆燃油中车的总载重能力为

$$10 \times 3000 + 60 \times 3000 + 50 \times 1500 = 285000 \text{ kg},$$

低于总需求 286140.35 kg，因此在这些较大容量车辆全部使用后仍至少需要 1 辆小车补足容量。最终使用 121 辆车，与载重下界基本一致。

表 3 问题一最终成本构成

指标	数值	单位
车辆启动成本	48400.00	元
油电能耗成本	30634.47	元
碳排放成本	6637.67	元
早到等待成本	262.35	元
迟到惩罚成本	187.86	元
总成本	86122.34	元
总距离	14574.78	km
总碳排放	10211.80	kg
车辆数	121	辆
总油耗	3839.76	L
总电耗	862.13	kW·h
总等待时间	13.12	h
总迟到时间	3.76	h

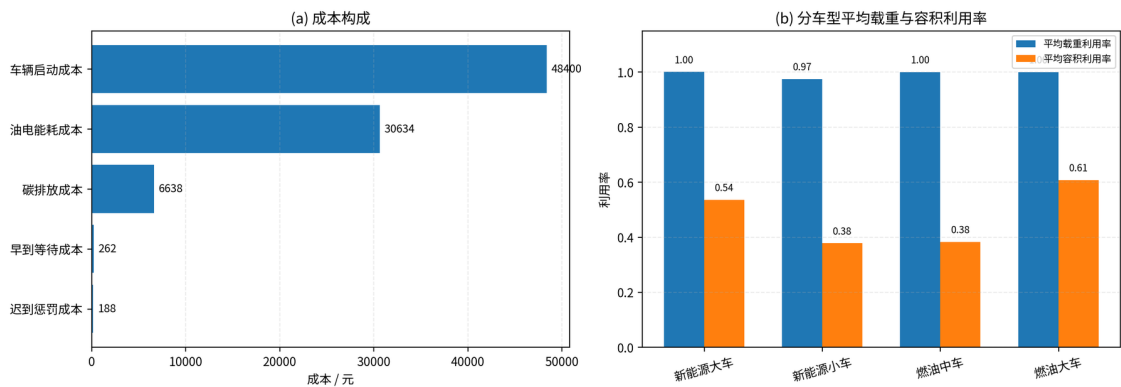


图 2 问题一最终方案成本构成与车型利用率综合分析

从表3和图2可见，车辆启动成本和油电能耗成本占主要部分；同时，燃油大车、燃油中车和新能源大车的平均载重利用率接近 100%。由于车辆数已接近容量下界，后续优化主要依赖于减少绕行、改善路线顺序和优化车型匹配。

6.3.2 车型使用结果

表 4 问题一最终车型使用与利用率

车辆类型	使用数量	总里程/km	总成本/元	总碳排/kg	载重利用率	容积利用率
新能源大车	10	2017.24	5711.93	402.72	1.0000	0.5352
新能源小车	1	149.20	520.04	29.21	0.9744	0.3776
燃油中车	50	5945.40	37086.01	4683.43	0.9992	0.3819
燃油大车	60	6462.93	42804.36	5096.44	0.9999	0.6072

燃油大车、燃油中车和新能源大车均被全部使用，另使用 1 辆新能源小车补足剩余重量需求。各车型平均载重利用率接近 1，说明本组数据主要受重量约束驱动；容积利用率相对较低，说明体积约束不是最终车辆数的主导因素。

6.3.3 算法递进效果

表 5 问题一算法对比结果

算法	总成本/元	车辆数	总距离/km	总碳排/kg	成本差距/%
确定性贪心构造	89096.21	121	15013.50	11119.41	3.45
GRASP + 2-opt	88108.11	121	14665.19	10768.93	2.31
GRASP + VND + 车型再分配	86122.34	121	14574.78	10211.80	0.00

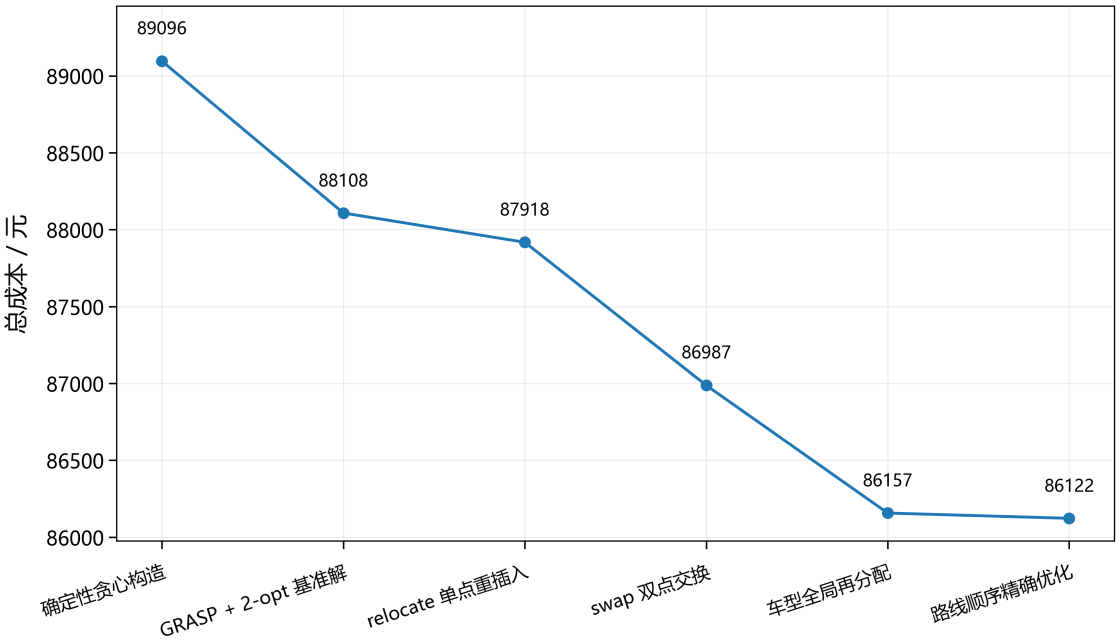


图 3 问题一算法升级过程中的成本下降曲线

由图3可见，在 GRASP+2-opt 基准解之后，relocate、swap、车型全局再分配和路线顺序精确优化依次压缩成本，其中车型全局再分配贡献最大，说明同一路线集合下的车型匹配对能耗和碳成本具有明显影响。相较 GRASP+2-opt 基准解，最终 GRASP-VND 方案总成本降低

$$88108.11 - 86122.34 = 1985.77 \text{ 元},$$

降幅为

$$\frac{1985.77}{88108.11} \times 100\% = 2.25\%.$$

同时，总里程减少 90.41 km，碳排放减少 557.14 kg，说明 VND 阶段不仅降低经济成本，也改善了能耗和碳排放表现。

6.3.4 路线和到达时间

表 6 问题一部分车辆路径示例

路线编号	车型	路径	发车-返回	距离/km	成本/元
1	燃油大车	0-76-66-0	09:15-13:12	95.14	671.06
2	燃油大车	0-66-79-0	09:10-13:40	126.77	754.10
3	燃油大车	0-16-79-85-11-6-0	08:00-14:09	133.01	801.39
4	新能源大车	0-37-82-94-75-0	09:20-16:33	205.20	562.43
5	新能源大车	0-35-87-32-75-0	08:00-16:33	207.00	591.33
6	燃油大车	0-88-8-0	09:55-13:22	75.28	623.19
7	燃油大车	0-52-0	13:30-18:01	178.18	906.42
8	燃油大车	0-59-0	09:25-14:16	164.15	870.26

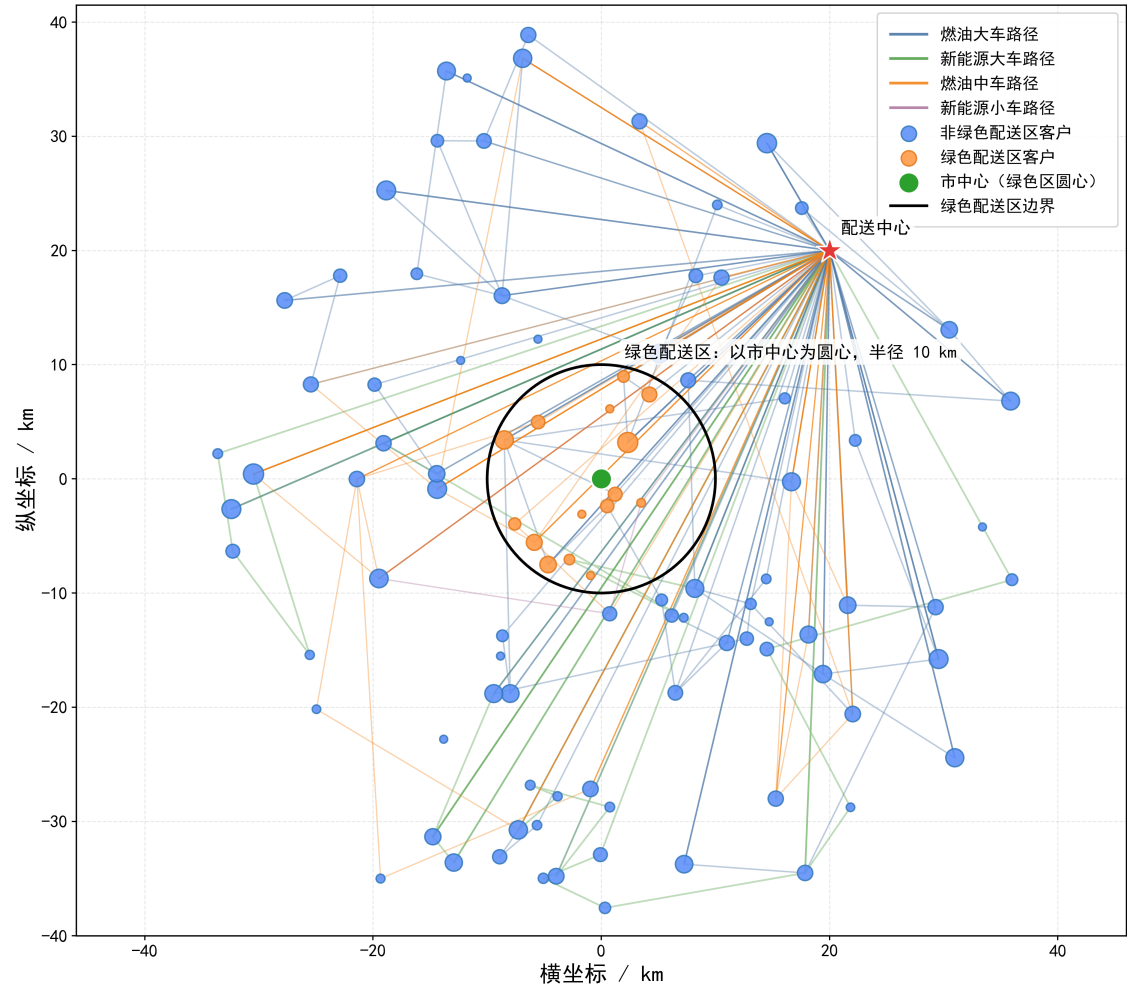


图 4 问题一最终调度路径总览

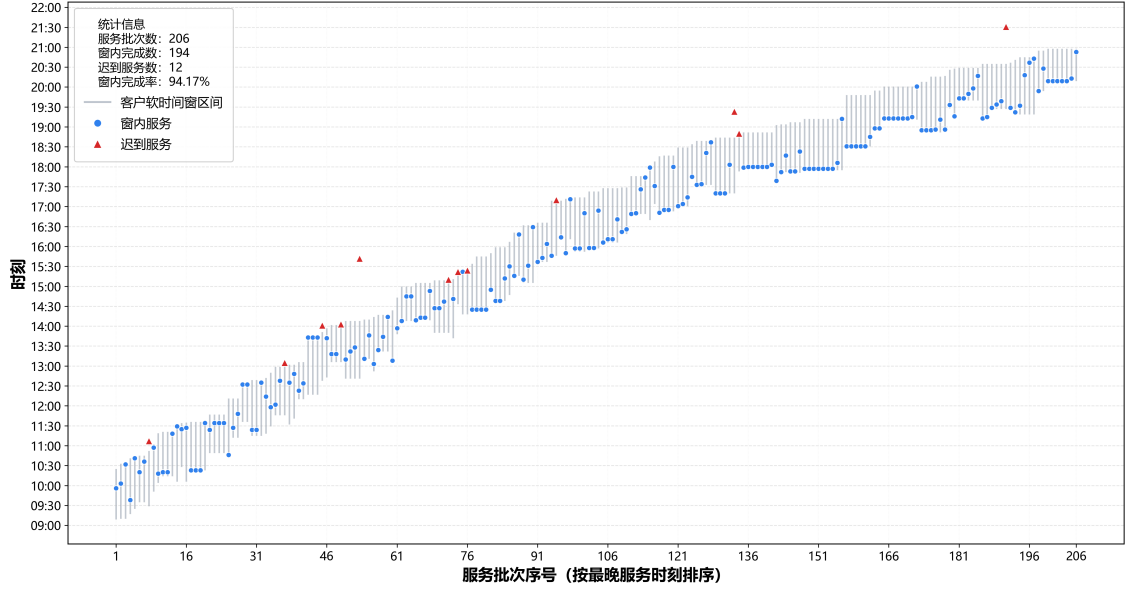


图 5 问题一客户软时间窗与最终服务时刻

最终服务记录共 206 条。由图5可见，大部分服务批次落在软时间窗内，窗内完成率为 94.17%；迟到服务批次较少，总迟到时间为 3.76 小时，迟到惩罚成本仅占总成本约 0.22%。

七、 问题二：环保政策影响下的车辆调度策略

7.1 绿色配送区限行模型

问题二加入如下政策：8:00–16:00 禁止燃油车进入绿色配送区。根据前文定义，绿色配送区客户集合为

$$G = \{i \in C \mid X_i^2 + Y_i^2 \leq 10^2\}. \quad (20)$$

令 $g_i = 1$ 表示客户 i 位于绿色区， $f_k = 1$ 表示车辆 k 为燃油车， $z_i^k = 1$ 表示车辆 k 服务客户 i 。以 8:00 为时间原点，16:00 对应 $a = 8$ 。若燃油车服务绿色区客户，则到达该客户的时间必须不早于 16:00：

$$a_i^k \geq 8 - M(1 - z_i^k) - M(1 - g_i) - M(1 - f_k). \quad (21)$$

在算法搜索过程中，加入政策违规惩罚项：

$$\min C_{\text{policy}} = C_{\text{total}} + \lambda_{\text{vio}} N_{\text{vio}}, \quad (22)$$

其中 N_{vio} 为燃油车在 16:00 前服务绿色区客户的次数， λ_{vio} 为足够大的惩罚系数。最终可行解必须满足

$$N_{\text{vio}} = 0. \quad (23)$$

7.2 政策感知 ALNS 算法

问题二不重新推翻问题一全部路线，而是以问题一最终方案为初始解，识别政策冲突并局部修复。本文设计的政策感知 ALNS 包括以下四类破坏与修复思想。

7.2.1 破坏算子

- (1) **绿色区冲突移除**：优先移除燃油车在 8:00–16:00 服务的绿色区客户。
- (2) **时间窗冲突移除**：移除等待成本或迟到成本较高的客户，释放局部时间结构。
- (3) **车型错配移除**：移除由不经济车型服务的客户，为车型再分配创造空间。
- (4) **随机移除**：随机移除少量客户，提高搜索多样性。

7.2.2 修复算子

- (1) **绿色优先插入**：绿色区客户优先插入新能源车路线；若插入燃油车路线，则要求服务时刻不早于 16:00。
- (2) **延后发车修复**：对于载重结构合理但到达绿色区过早的燃油车路线，延后发车使其到达绿色区客户时刻满足政策。
- (3) **路线内部重排序**：改变同一路线内客户访问顺序，降低延后发车造成的时间窗损失。
- (4) **车型全局再分配**：在路线结构基本确定后，重新匹配路线与车型，优先让新能源车承担绿色区早间服务任务。

问题二政策感知 ALNS 求解流程

- Step1: 读取问题一最终路线作为初始解。
- Step2: 扫描所有路线，标记燃油车 16:00 前服务绿色区客户的违规访问点。
- Step3: 按自适应权重选择破坏算子，移除一组受政策影响的客户或路线片段。
- Step4: 按绿色优先插入、延后发车、路线重排序和车型再分配修复路线。
- Step5: 若新解可行且成本更低则接受；若成本略高，则按模拟退火准则以一定概率接受。
- Step6: 当 $N_{vio} = 0$ 且成本稳定时，输出政策约束下的最终方案。

7.3 问题二结果分析

7.3.1 政策冲突消除效果

问题一基准方案中，燃油车 16:00 前服务绿色区客户的违规访问次数为 16 次，涉及绿色区客户 7 个。经政策感知 ALNS 修复后，问题二最终方案的违规访问次数降为 0。图6直观展示了政策前后违规点的变化。

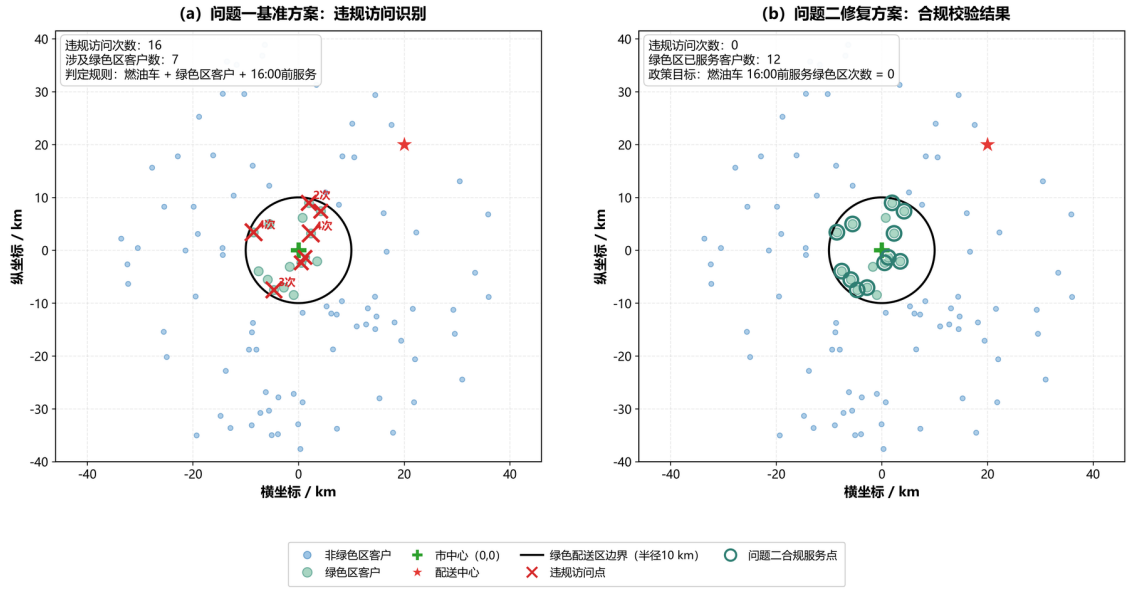


图 6 绿色限行政策前后违规访问点对比

图7给出受政策影响的代表性路线修复过程。对于问题一中由燃油车提前服务绿色区客户的路线，问题二通过新能源替代、延后服务和局部重排序消除违规。

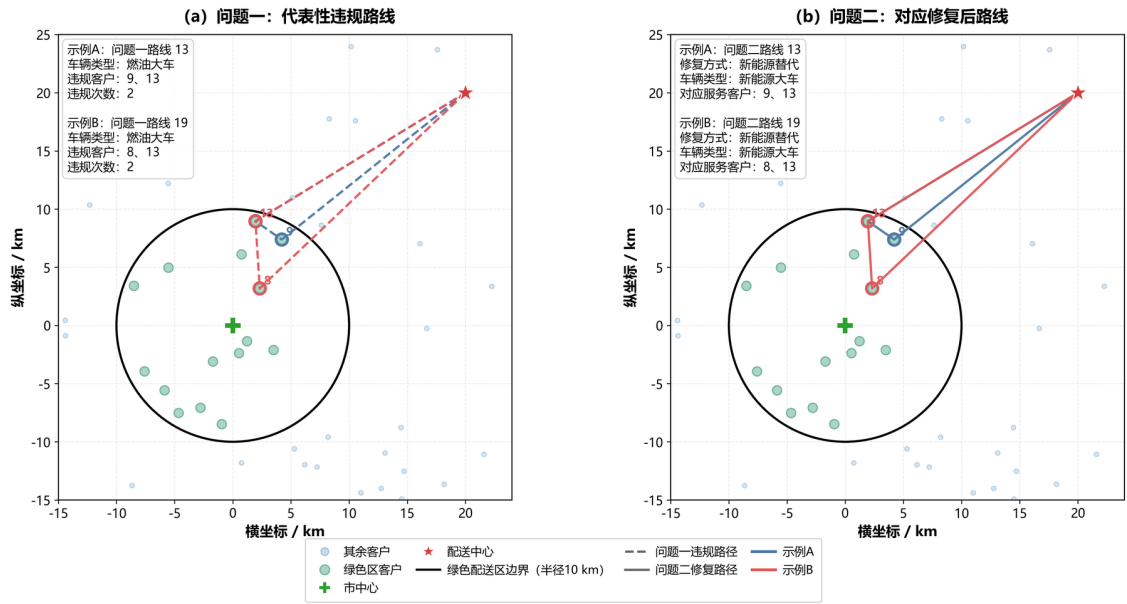


图 7 受政策影响代表性路线修复前后对比

7.3.2 成本与碳排放变化

问题二最终调度方案仍使用 121 辆车，其中新能源车 11 辆、燃油车 110 辆，车辆使用结构与问题一保持一致。最终总成本为 87957.53 元，总里程为 14558.40 km，总碳排放为 10481.14 kg。

表 7 问题二最终成本构成

指标	数值	单位
车辆启动成本	48400.00	元
油电能耗成本	31411.38	元
碳排放成本	6812.74	元
早到等待成本	237.86	元
迟到惩罚成本	1095.54	元
总成本	87957.53	元
总里程	14558.40	km
总碳排放	10481.14	kg
使用车辆数	121	辆
政策违规次数	0	次

表 8 问题一与问题二关键指标对比

指标	问题一	问题二	变化量	变化率	单位
总成本	86122.34	87957.53	1835.19	2.13%	元
车辆数	121	121	0	0.00%	辆
总里程	14574.78	14558.40	-16.38	-0.11%	km
总碳排放	10211.80	10481.14	269.35	2.64%	kg
迟到惩罚成本	187.86	1095.54	907.68	483.17%	元
燃油车 16 点前服务绿色区次数	16	0	-16	-100.00%	次

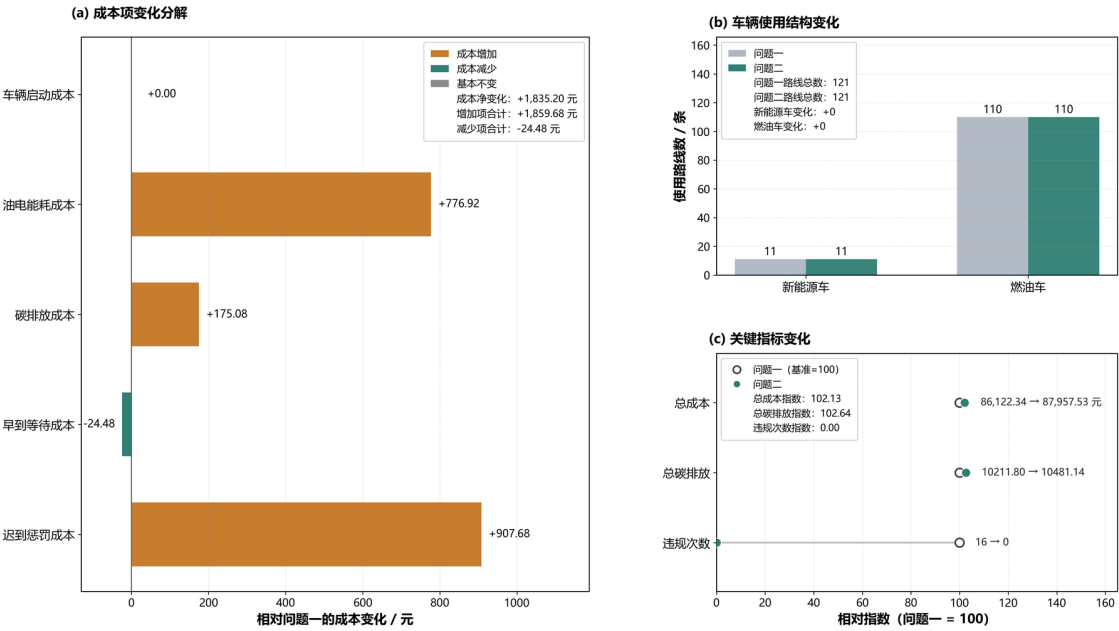


图 8 绿色限行政策对成本、车型结构和关键指标的影响

由表8可知，政策实施后总成本增加 1835.19 元，主要来自油电能耗成本、碳排放成本和迟到惩罚成本上升。其中迟到惩罚成本增加较明显，是因为部分燃油车服务绿色区客户必须推迟至 16:00 之后。总里程略有下降，但总碳排放反而上升，说明单纯区域限行不一定带来系统层面的碳排放降低；若要进一步实现减排，需要配套提高碳价、增加新能源车供给或设置碳排放上限。

7.3.3 ALNS 修复过程与绿色区服务时刻校验

图9展示了政策冲突修复过程。初始方案存在 16 次违规，经过延后发车修复、路线重排序和车型再分配后，违规次数最终为 0，总成本稳定在 87957.53 元。

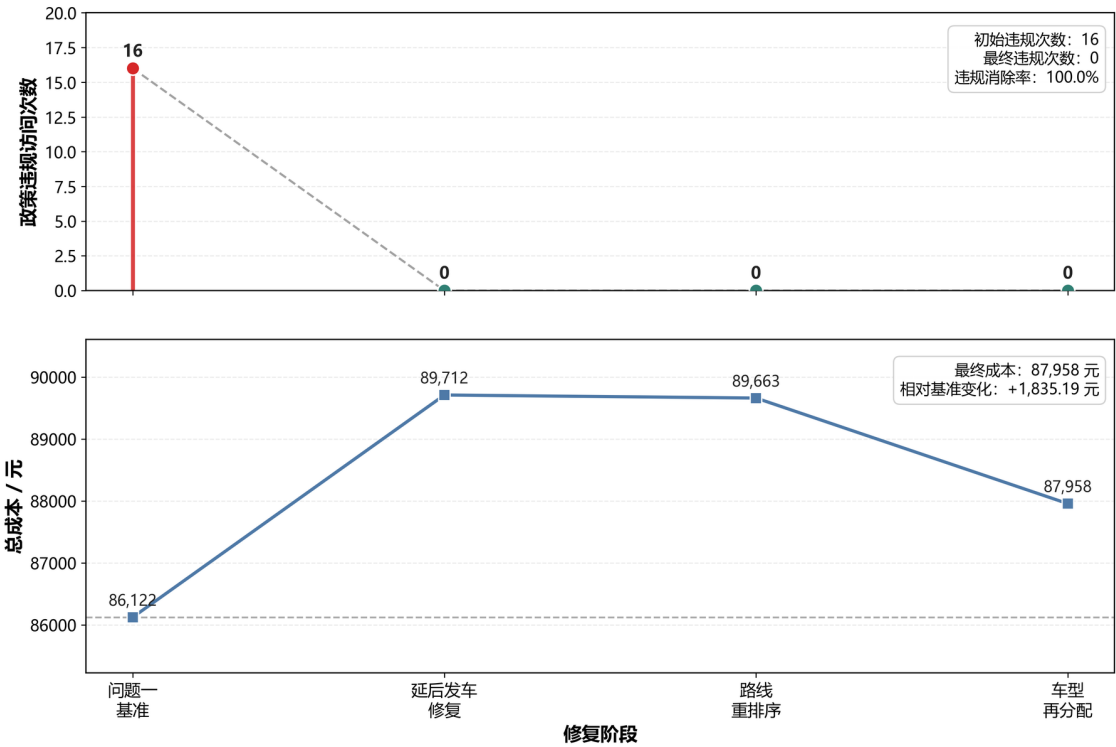


图 9 政策违规访问次数消除过程

为从时间维度验证政策合规性，图10给出了绿色区客户时间窗与最终服务时刻。可以看出，新能源车可以在 16:00 前服务绿色区客户，而燃油车涉及绿色区客户的服务均安排在 16:00 后，因此最终方案满足限行政策。

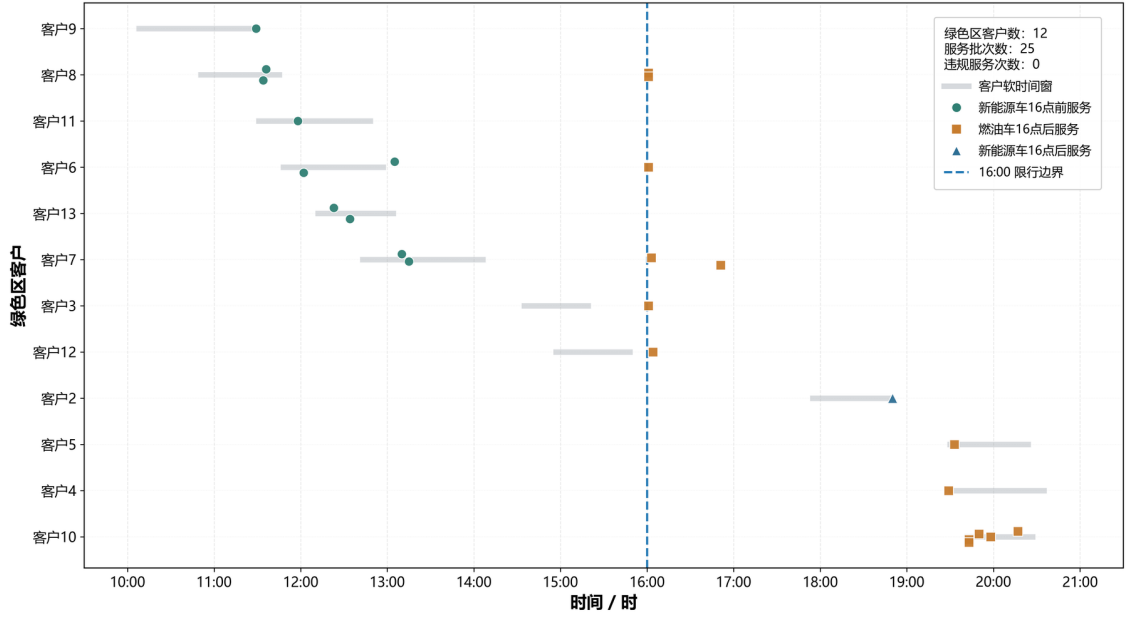


图 10 绿色区客户时间窗与最终服务时刻甘特图

7.3.4 问题二部分路径示例

表 9 问题二部分车辆路径示例

路线编号	车型	路径	发车-返回	距离/km	成本/元
3	新能源大车	0-16-79-85-11-6-0	08:00-14:09	133.01	512.08
6	新能源大车	0-88-8-0	10:00-13:24	75.28	459.94
12	新能源大车	0-33-42-7-0	09:25-14:37	149.66	519.34
13	新能源大车	0-9-13-0	10:20-13:42	87.44	467.87
16	燃油大车	0-8-0	15:05-17:17	66.05	801.94
27	燃油大车	0-6-0	14:55-17:17	81.28	839.06
28	燃油大车	0-30-7-0	14:40-17:55	86.89	844.97
37	燃油大车	0-12-7-0	15:10-18:08	72.54	758.67

八、 问题三：动态事件下的实时车辆调度策略

8.1 滚动时域动态调度模型

在问题二最终合规方案基础上，问题三进一步考虑配送过程中的订单取消、新增订单和时间窗调整。与重新规划所有车辆路径不同，本文采用事件触发的滚动时域动态调度模型：当事件在时刻 τ 发生时，冻结已完成任务和已确认在途任务，仅对未执行任务与受事件影响路线进行局部重优化。

设车辆 k 在事件时刻 τ 的状态为

$$S_k(\tau) = (p_k(\tau), t_k(\tau), q_k^w(\tau), q_k^v(\tau), R_k^{\text{done}}, R_k^{\text{remain}}, \text{type}_k), \quad (24)$$

其中 $p_k(\tau)$ 表示当前位置, $t_k(\tau)$ 表示当前可继续执行的时刻, $q_k^w(\tau)$ 与 $q_k^v(\tau)$ 分别表示剩余载重与剩余容积, R_k^{done} 和 R_k^{remain} 分别表示已完成任务集合和待执行任务集合。动态调度的目标函数写为

$$\min C^{\text{dyn}} = C_{\text{remain}} + C_{\text{adjust}}, \quad (25)$$

其中 C_{remain} 仍由车辆启动成本、油电能耗成本、碳排放成本、早到等待成本和迟到惩罚成本组成; C_{adjust} 用于衡量方案扰动程度, 本文主要用受影响路线数、撤销路线数、新增路线数和调整路线比例进行评价。

为保证执行稳定性, 动态调度需要满足三类约束。首先, 冻结约束要求

$$R_k^{\text{done}}(\tau) \cap R_k^{\text{new}} = \emptyset, \quad (26)$$

即已完成服务不得重新分配; 其次, 车辆剩余容量必须满足

$$\sum_{i \in R_k^{\text{new}}} q_i^k \leq q_k^w(\tau), \quad \sum_{i \in R_k^{\text{new}}} r_i^k \leq q_k^v(\tau). \quad (27)$$

最后, 若客户位于绿色配送区且由燃油车服务, 则仍必须满足式中所示的政策约束。

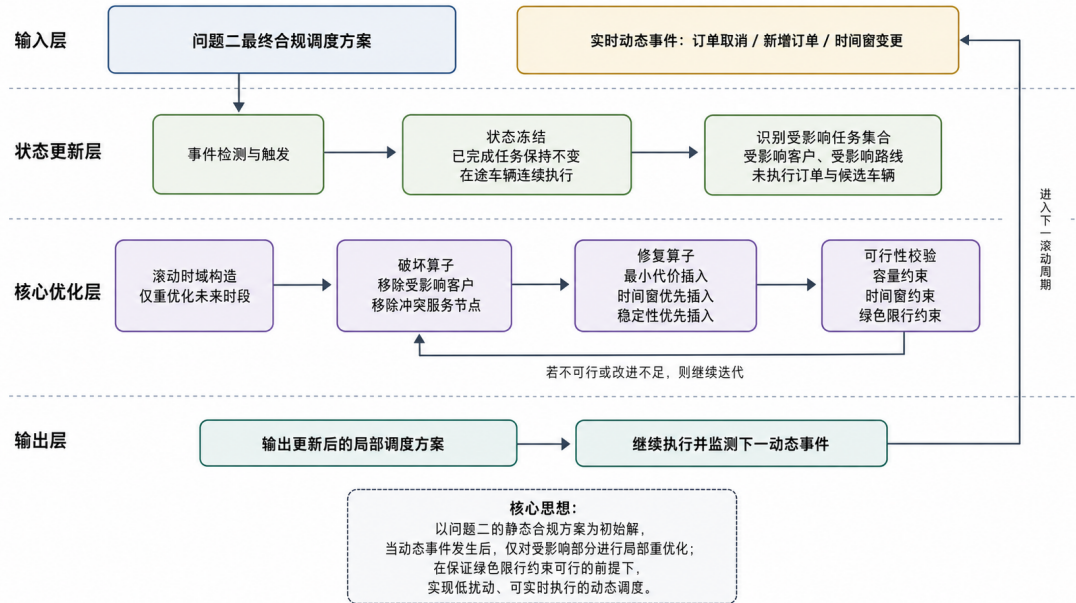


图 11 滚动时域动态调度框架

8.2 滚动时域 ALNS 求解策略

问题三以问题二最终合规方案作为初始计划, 将政策感知 ALNS 嵌入滚动时域框架。事件发生后, 算法首先识别受影响任务集合, 再根据事件类型调用相应破坏与修复策略。其核心流程如下:

问题三 RH-ALNS 求解流程

Step1: 读取问题二最终合规方案, 并按照事件发生时刻更新车辆状态。

Step2: 冻结已经完成或正在执行的任务, 识别受事件影响的客户、路线和候选车辆。

Step3: 若发生订单取消, 则删除对应未出发任务; 若发生新增订单, 则加入新增客户并选择候选新能源车; 若发生时间窗调整, 则抽取受影响路线。

Step4: 在滚动时域窗口内采用 ALNS 执行破坏和修复, 优先使用最小代价插入、时间窗优先插入和稳定性优先插入。

Step5: 重新校验容量约束、软时间窗约束和绿色限行约束, 若不满足则继续局部迭代。

Step6: 输出更新后的局部调度方案, 并继续监测下一动态事件。

其中, 订单取消事件对应“删除未执行任务”算子; 新增订单对应“待命车辆插入”算子; 时间窗收紧对应“发车时刻平移与局部时间修复”算子。该设计避免了全局重排, 因而更符合企业实际调度中的低扰动原则。

8.3 动态事件设置与响应结果

为验证模型对典型突发事件的响应能力, 本文设置三个代表性事件, 分别对应订单取消、新增绿色区订单和时间窗收紧。事件设定与局部响应结果见表10。

表 10 问题三动态事件设定与响应结果

事件	时刻	类型	事件描述	优化动作	影响路线	成本变化/元	碳排变化/kg
E1	10:30	订单取消	客户 61 的一个未出发服务批次被取消	撤销尚未出发的 Route 30	1	-728.52	-90.31
E2	11:00	新增订单	新增绿色区客户 101, 时间窗为 13:00–15:00	调用待命新能源车新增 Route 122	1	440.94	10.43
E3	12:00	时间窗调整	客户 56 最晚服务时刻由 15:09 提前至 14:10	将 Route 43 与 44 发车时刻整体提前 20 分钟	2	0.00	0.00

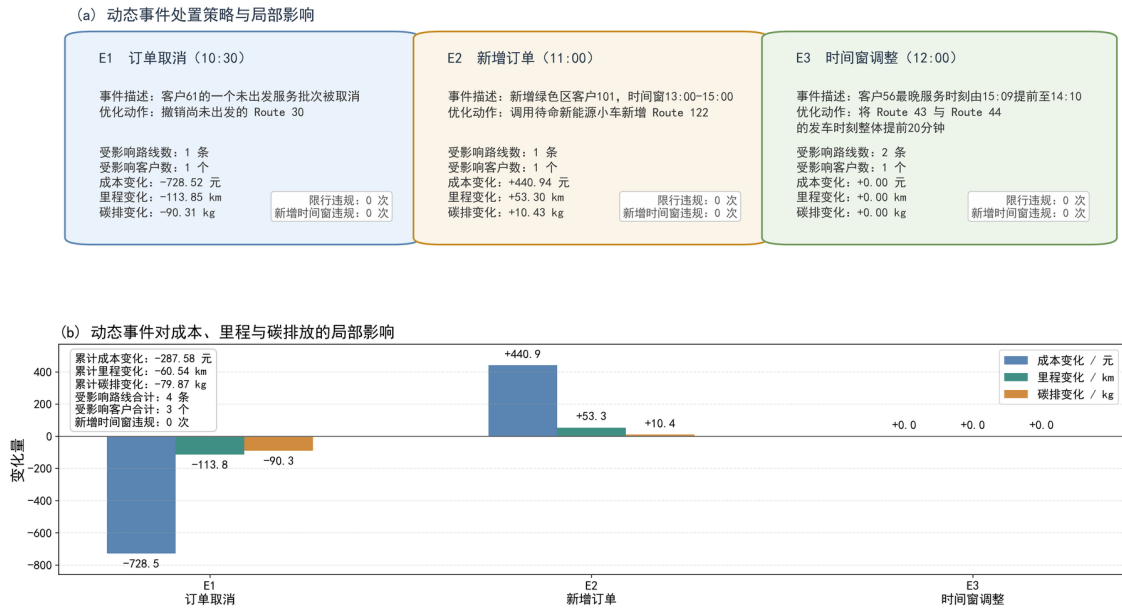


图 12 动态事件处置策略与局部影响汇总

由表10和图12可见，三类事件的处理方式不同。订单取消直接撤销未出发路线，从而降低成本和碳排放；新增订单需要增加一条新能源小车紧急路线，因此成本和碳排放上升；时间窗调整只改变发车时刻而不改变路径结构，因此成本、里程和碳排放均不变。

8.4 事件 E2：新增绿色区订单插入

事件 E2 在 11:00 触发，新增客户 101 位于绿色配送区内，服务时间窗为 13:00–15:00。由于绿色区在 16:00 前禁止燃油车服务，本文调用待命新能源小车新增 Route 122，路径为 0–101–0，发车时刻为 12:20，返回时刻为 14:11。该处理方式不改变原有路线结构，只新增一条紧急补充路线。

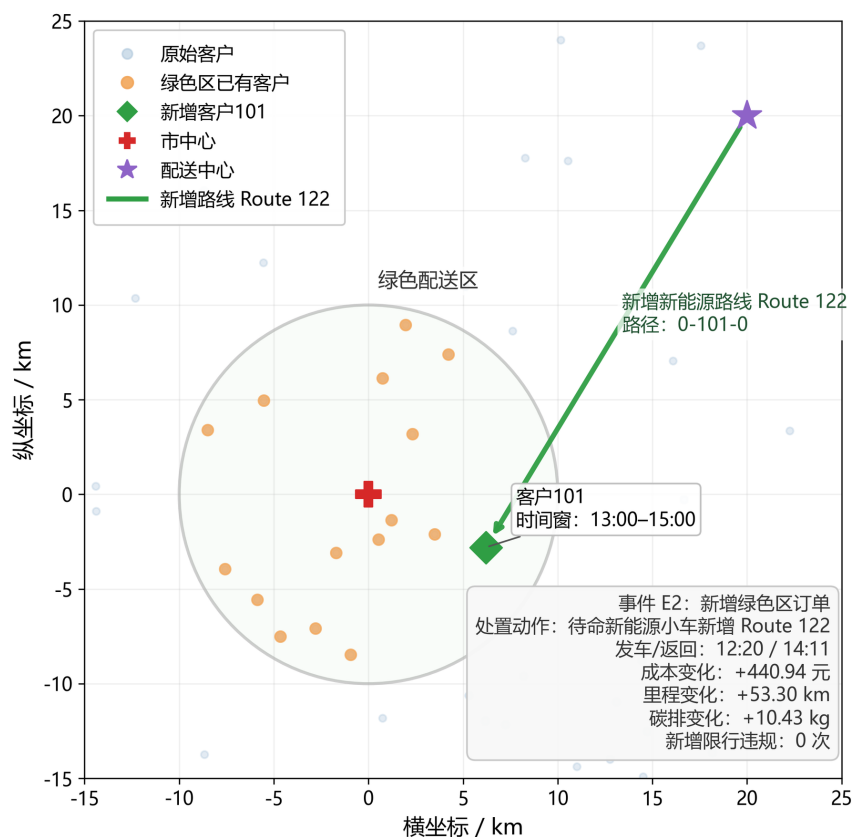


图 13 事件 E2 新增绿色区订单插入示意

Route 122 的新增里程为 53.30 km，成本增加 440.94 元，碳排放增加 10.43 kg，新增限行违规次数为 0。这说明在绿色限行政策下，预留少量新能源车辆可显著提高系统对绿色区临时订单的响应能力。

8.5 事件 E3：时间窗收紧修复

事件 E3 中，客户 56 的最晚服务时刻由 15:09 提前至 14:10。问题二原方案中 Route 43 和 Route 44 对该客户的服务时刻为 14:27，不再满足收紧后的时间窗。由于这两条路线在事件发生时尚未发车，本文将两条路线发车时刻由 13:00 整体提前至 12:40，调整后服务时刻为 14:07，重新满足最晚服务时刻要求。

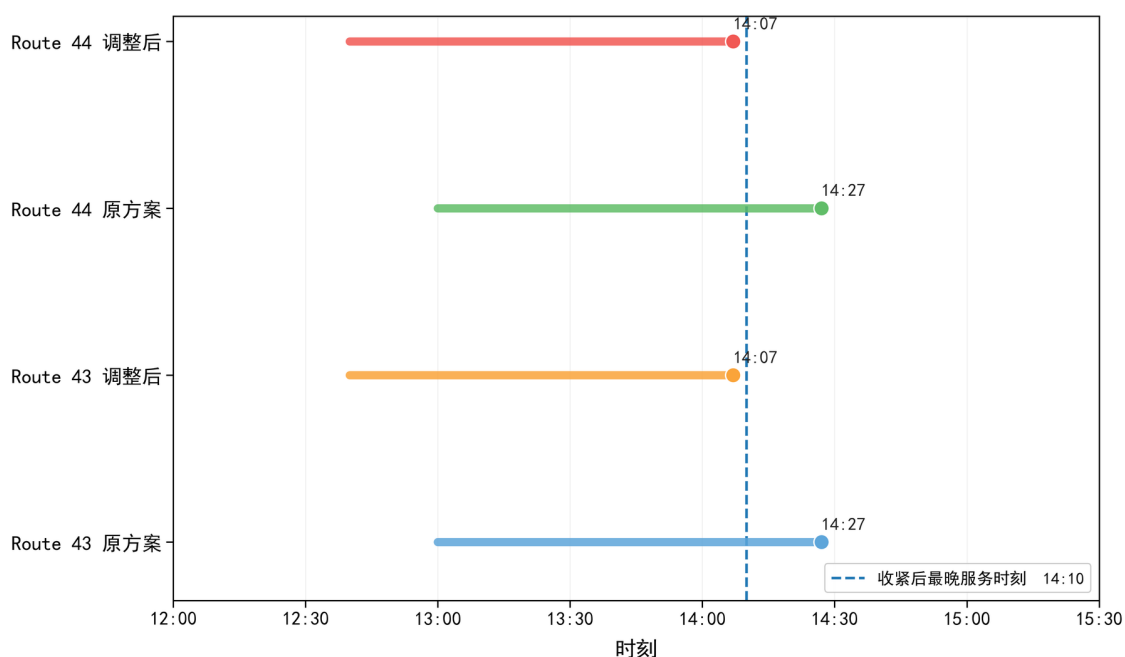


图 14 事件 E3 时间窗收紧后的服务时刻调整

需要说明的是，E3 的修复仅改变发车时刻，不改变客户分配、访问顺序、车辆类型与行驶距离。因此在本文成本核算口径下，其成本变化、里程变化与碳排放变化均为 0。若进一步考虑拥堵时段速度微观变化或司机工时约束，则该事件可能产生更细粒度的时间成本。

8.6 问题三最终结果

三次动态事件累计作用后，问题三最终总成本为 87669.94 元，总里程为 14497.85 km，总碳排放为 10401.27 kg，限行违规次数仍为 0。与问题二基准方案对比如表11。

表 11 问题二与问题三关键指标对比

指标	问题二	问题三	变化量	变化率
总成本/元	87957.53	87669.94	-287.58	-0.33%
总里程/km	14558.40	14497.85	-60.54	-0.42%
总碳排放/kg	10481.14	10401.27	-79.87	-0.76%
服务路线数/条	121	121	0	0.00%
限行违规次数/次	0	0	0	0.00%

表 12 问题三调度稳定性指标

指标	数值	单位
总路线数	121	条
发生调整的路线数	4	条
路线调整比例	3.31	%
新增路线数	1	条
撤销路线数	1	条
仅修改发车时刻的路线数	2	条
受影响客户数	3	个
最终限行违规次数	0	次
动态事件新增时间窗违规次数	0	次

由表11和表12可见，三次事件共影响 4 条路线，占总路线数 3.31%。最终方案仍保持政策完全合规，且总成本相较问题二基准略有下降，主要原因是 E1 撤销 Route 30 带来的成本节约大于 E2 新增紧急订单带来的成本增加。该结果表明，滚动时域 ALNS 能在局部扰动较小的条件下完成动态调度调整。

九、 模型评价与进一步改进

9.1 模型优点

- (1) **模型主线与算法链条匹配度高。**本文没有把三问割裂处理，而是将问题一的静态可行解作为底座，将问题二的绿色限行约束作为定向修复环节，再把问题二的合规方案作为问题三滚动调度的初始计划，形成“静态构造-政策修复-动态响应”的递进优化链。该结构使变量定义、成本核算、路线结果和算法输出能够逐层复用，既突出 GRASP-VND、政策感知 ALNS 与 RH-ALNS 的分工，也降低了重复建模带来的逻辑断裂。
- (2) **成本刻画较完整。**目标函数同时考虑车辆启动、油电能耗、碳排放、早到等待和迟到惩罚，能够较好反映物流企业实际经营成本，并能解释政策实施后成本上升的来源。
- (3) **约束处理贴合数据条件。**在缺少完整道路轨迹的情况下，将绿色限行转化为客户服务节点约束，既避免了主观判断车辆是否穿越绿色区，又保留了政策合规性校验的核心信息。
- (4) **算法具有针对性。**问题一采用 GRASP-VND 改善静态路径质量；问题二采用政策感知 ALNS 定向修复绿色区限行冲突；问题三将 ALNS 嵌入滚动时域框架，强调低扰动动态响应。
- (5) **结果可复现、可核验。**项目保留了路线汇总、到达时间明细、成本汇总、事件响应表和完整求解代码，表格指标能够与程序输出相互核对。

9.2 模型不足

- (1) 本文将绿色限行解释为节点服务约束，未判断道路行驶过程中是否穿越绿色区；这是由于附件未提供完整道路轨迹。
- (2) GRASP-VND、ALNS 和 RH-ALNS 均为启发式算法，不能严格证明全局最优，结果会受邻域顺序和参数设置影响。
- (3) 当前模型未加入新能源车续航、充电排队、司机工时和道路拥堵实时更新等约束，后续可进一步扩展。

9.3 政策含义与改进方向

问题二结果表明，绿色限行政策能有效消除燃油车在限行时段服务绿色区客户的行为，但在新能源车数量有限、碳价较低条件下，系统总碳排放不一定下降。原因在于部分燃油车服务被迫延后，导致时间窗惩罚和高载重燃油车能耗增加。因此，若政策目标不仅是区域减污，还包括全局减碳，则应配合增加新能源车供给、提高碳排放成本权重或设置企业日碳排上限。

从实际部署角度看，物流企业可将本文模型嵌入日常调度系统：早晨先基于订单池生成静态合规计划；配送过程中持续监测取消、插单和时间窗变更事件；每次事件触发后仅对受影响车辆和未执行任务调用 RH-ALNS 快速重优化，从而兼顾效率、合规和执行稳定性。

参考文献

- [1] 第十八届“华中杯”大学生数学建模挑战赛组委会. A 题：城市绿色物流配送调度 [Z]. 2026.
- [2] 曹庆奎, 杨凯文, 任向阳, 赵丽飞. 基于交通流的多模糊时间窗车辆路径优化 [J]. 运筹与管理, 2018, 27(8): 20–26.
- [3] Solomon M M. Algorithms for the vehicle routing and scheduling problems with time window constraints[J]. Operations Research, 1987, 35(2): 254–265.
- [4] Feo T A, Resende M G C. Greedy randomized adaptive search procedures[J]. Journal of Global Optimization, 1995, 6(2): 109–133.
- [5] Hansen P, Mladenović N. Variable neighborhood search: Principles and applications[J]. European Journal of Operational Research, 2001, 130(3): 449–467.
- [6] Ropke S, Pisinger D. An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows[J]. Transportation Science, 2006, 40(4): 455–472.
- [7] ChatGPT, GPT-5.5 Pro, OpenAI, 2026-04-26.

附录 A 支撑材料文件列表

表 13 论文项目主要文件列表

文件或目录	功能说明
main.tex	论文主文件，包含摘要、正文、参考文献和附录。
figures/q1_*.png	问题一客户分布、路径总览、成本构成、算法收敛和时间窗图。
figures/q2_*.png	问题二限行违规对比、路线修复、政策影响、冲突消除和绿色区服务时刻图。
figures/q3_*.png	问题三动态调度框架、事件汇总、新增订单插入和时间窗收紧修复图。
data/problem1_*.csv	问题一清洗数据、路线结果、成本汇总、到达时间明细、绿色区核验和 VND 收敛记录。
data/problem2_*.csv	问题二成本汇总、政策前后对比、路线结果、到达时间明细和校验摘要。
data/problem3_*.csv	问题三事件设定、局部调整、成本汇总、稳定性指标和最终路线结果。
code/problem1_solver_base.py	问题一贪心基准求解程序。
code/problem1_grasp_vnd_solver.py	问题一 GRASP-VND 求解程序。
code/problem2_policy_alns_solver.py	问题二政策感知 ALNS 求解程序。
code/problem3_rh_alns_solver.py	问题三滚动时域 RH-ALNS 求解程序。

附录 B 源程序代码

B.1 问题一贪心基准求解程序：problem1_solver_base.py

```
1  # -*- coding: utf-8 -*-
2  """
3  问题一：静态环境下的城市绿色物流配送调度求解器（第一版）
4  模型类型：带软时间窗、时变速度、油/电耗与碳排放成本的异构车队拆分配送 VRP
5  输入：Cleaned_Customer_Nodes.csv, Cleaned_Distance_Matrix.csv
6  输出：路线汇总、节点到达明细、成本汇总和基础图表
7  """
8
9  import os
10 import math
11 import random
12 from dataclasses import dataclass, field
13 from typing import Dict, List, Tuple
14
15 import numpy as np
16 import pandas as pd
17 import matplotlib.pyplot as plt
18
19
20 BASE_DIR = "/mnt/data"
21 OUT_DIR = "/mnt/data/problem1_outputs"
22 os.makedirs(OUT_DIR, exist_ok=True)
23
24 WAIT_COST_PER_HOUR = 20.0
25 LATE_COST_PER_HOUR = 50.0
26 CARBON_PRICE = 0.65
27 FUEL_PRICE = 7.61
28 EV_PRICE = 1.64
29 FUEL_CARBON_FACTOR = 2.547
30 EV_CARBON_FACTOR = 0.501
31 START_COST = 400.0
32 SERVICE_TIME_HR = 1.0 / 3.0
33
34 EPS = 1e-7
35
36
37 @dataclass
38 class VehicleType:
39     type_id: int
40     name: str
41     max_weight: float
42     max_volume: float
43     count_limit: int
44     is_ev: bool
45
46
47 @dataclass
48 class Route:
49     route_id: int
50     vehicle: VehicleType
51     stops: List[int] = field(default_factory=list)
52     deliveries_w: Dict[int, float] = field(default_factory=dict)
53     deliveries_v: Dict[int, float] = field(default_factory=dict)
54     metrics: Dict[str, float] = field(default_factory=dict)
55
56
57 def hour_to_clock(h: float) -> str:
58     total_minutes = int(round(8 * 60 + h * 60))
```

```

59     total_minutes %= 24 * 60
60     return f"{total_minutes // 60:02d}:{total_minutes % 60:02d}"
61
62
63 def get_traffic_info(t: float) -> Tuple[float, float, float]:
64     """
65     t 为从 8:00 起算的小时数。
66     返回: 速度均值、速度方差、当前交通状态结束时刻。
67     """
68     if 0.0 <= t < 1.0:
69         return 9.8, 4.7 ** 2, 1.0
70     if 1.0 <= t < 2.0:
71         return 55.3, 0.1 ** 2, 2.0
72     if 2.0 <= t < 3.5:
73         return 35.4, 5.2 ** 2, 3.5
74     if 3.5 <= t < 5.0:
75         return 9.8, 4.7 ** 2, 5.0
76     if 5.0 <= t < 7.0:
77         return 55.3, 0.1 ** 2, 7.0
78     if 7.0 <= t < 9.0:
79         return 35.4, 5.2 ** 2, 9.0
80     return 35.4, 5.2 ** 2, float("inf")
81
82
83 def energy_per_100km(v_mean: float, v_var: float, load_ratio: float, is_ev: bool) -> float:
84     ev2 = v_mean * v_mean + v_var
85     load_ratio = max(0.0, min(1.0, load_ratio))
86     if is_ev:
87         base = 0.0014 * ev2 - 0.12 * v_mean + 36.19
88         return base * (1.0 + 0.35 * load_ratio)
89     base = 0.0025 * ev2 - 0.2554 * v_mean + 31.75
90     return base * (1.0 + 0.40 * load_ratio)
91
92
93 def travel_and_cost(distance_km: float, start_t: float, load_ratio: float, is_ev: bool) -> Dict[str,
↵ float]:
94     """
95     按交通时段切分一条边, 返回旅行时间、油/电耗、碳排和能耗碳成本。
96     """
97     if distance_km <= EPS:
98         return {
99             "travel_time": 0.0, "energy_cost": 0.0, "carbon_cost": 0.0,
100             "carbon_kg": 0.0, "fuel_l": 0.0, "electric_kwh": 0.0
101         }
102
103     remaining = distance_km
104     t = start_t
105     travel_time = 0.0
106     energy_cost = 0.0
107     carbon_cost = 0.0
108     carbon_kg = 0.0
109     fuel_l = 0.0
110     electric_kwh = 0.0
111
112     guard = 0
113     while remaining > EPS and guard < 30:
114         guard += 1
115         v_mean, v_var, t_end = get_traffic_info(t)
116         available_time = max(0.0, t_end - t) if math.isfinite(t_end) else float("inf")
117         possible_dist = v_mean * available_time if math.isfinite(available_time) else float("inf")
118         if possible_dist >= remaining - EPS:
119             seg_dist = remaining
120             seg_time = seg_dist / v_mean

```

```

121         else:
122             seg_dist = possible_dist
123             seg_time = available_time
124
125         per100 = energy_per_100km(v_mean, v_var, load_ratio, is_ev)
126         energy = (seg_dist / 100.0) * per100
127         if is_ev:
128             electric_kwh += energy
129             energy_cost += energy * EV_PRICE
130             carbon = energy * EV_CARBON_FACTOR
131         else:
132             fuel_l += energy
133             energy_cost += energy * FUEL_PRICE
134             carbon = energy * FUEL_CARBON_FACTOR
135         carbon_kg += carbon
136         carbon_cost += carbon * CARBON_PRICE
137
138         travel_time += seg_time
139         t += seg_time
140         remaining -= seg_dist
141
142         if seg_time <= EPS and remaining > EPS:
143             t = t_end + EPS
144
145     return {
146         "travel_time": travel_time,
147         "energy_cost": energy_cost,
148         "carbon_cost": carbon_cost,
149         "carbon_kg": carbon_kg,
150         "fuel_l": fuel_l,
151         "electric_kwh": electric_kwh
152     }
153
154
155 def evaluate_route(route: Route, dist_mat: np.ndarray, ready: np.ndarray, due: np.ndarray) ->
↳ Tuple[Dict[str, float], List[Dict]]:
156     vehicle = route.vehicle
157     current_node = 0
158     current_t = 0.0
159     remaining_w = sum(route.deliveries_w.values())
160     total_w = remaining_w
161     total_v = sum(route.deliveries_v.values())
162
163     fixed_cost = START_COST
164     energy_cost = 0.0
165     carbon_cost = 0.0
166     wait_cost = 0.0
167     late_cost = 0.0
168     distance = 0.0
169     travel_time = 0.0
170     wait_time = 0.0
171     late_time = 0.0
172     fuel_l = 0.0
173     electric_kwh = 0.0
174     carbon_kg = 0.0
175
176     detail_rows = []
177     sequence = 0
178
179     for node in route.stops:
180         leg_dist = float(dist_mat[current_node, node])
181         load_ratio = remaining_w / vehicle.max_weight if vehicle.max_weight > 0 else 0.0
182         leg = travel_and_cost(leg_dist, current_t, load_ratio, vehicle.is_ev)

```

```

183     arrival_before_wait = current_t + leg["travel_time"]
184
185     this_wait = max(0.0, ready[node] - arrival_before_wait)
186     service_start = max(arrival_before_wait, ready[node])
187     this_late = max(0.0, arrival_before_wait - due[node])
188     departure = service_start + SERVICE_TIME_HR
189
190     distance += leg_dist
191     travel_time += leg["travel_time"]
192     energy_cost += leg["energy_cost"]
193     carbon_cost += leg["carbon_cost"]
194     carbon_kg += leg["carbon_kg"]
195     fuel_l += leg["fuel_l"]
196     electric_kwh += leg["electric_kwh"]
197     wait_time += this_wait
198     late_time += this_late
199     wait_cost += this_wait * WAIT_COST_PER_HOUR
200     late_cost += this_late * LATE_COST_PER_HOUR
201
202     sequence += 1
203     delivered_w = route.deliveries_w[node]
204     delivered_v = route.deliveries_v[node]
205     detail_rows.append({
206         "Route_ID": route.route_id,
207         "Vehicle_Type": vehicle.name,
208         "Seq": sequence,
209         "Node_ID": node,
210         "Arrive_Hr_From_8": arrival_before_wait,
211         "Arrive_Time": hour_to_clock(arrival_before_wait),
212         "Service_Start_Hr_From_8": service_start,
213         "Service_Start_Time": hour_to_clock(service_start),
214         "Departure_Hr_From_8": departure,
215         "Departure_Time": hour_to_clock(departure),
216         "Ready_Hr_From_8": ready[node],
217         "Due_Hr_From_8": due[node],
218         "Wait_Hr": this_wait,
219         "Late_Hr": this_late,
220         "Delivered_Weight_kg": delivered_w,
221         "Delivered_Volume_m3": delivered_v,
222         "Leg_Distance_km": leg_dist,
223         "Remaining_Load_Before_Service_kg": remaining_w
224     })
225
226     remaining_w -= delivered_w
227     current_node = node
228     current_t = departure
229
230     leg_dist = float(dist_mat[current_node, 0])
231     load_ratio = max(0.0, remaining_w / vehicle.max_weight)
232     leg = travel_and_cost(leg_dist, current_t, load_ratio, vehicle.is_ev)
233     distance += leg_dist
234     travel_time += leg["travel_time"]
235     energy_cost += leg["energy_cost"]
236     carbon_cost += leg["carbon_cost"]
237     carbon_kg += leg["carbon_kg"]
238     fuel_l += leg["fuel_l"]
239     electric_kwh += leg["electric_kwh"]
240     end_t = current_t + leg["travel_time"]
241
242     total_cost = fixed_cost + energy_cost + carbon_cost + wait_cost + late_cost
243
244     metrics = {
245         "route_id": route.route_id,

```

```

246         "vehicle_type": vehicle.name,
247         "vehicle_is_ev": int(vehicle.is_ev),
248         "stops": len(route.stops),
249         "distance_km": distance,
250         "total_weight_kg": total_w,
251         "total_volume_m3": total_v,
252         "weight_utilization": total_w / vehicle.max_weight,
253         "volume_utilization": total_v / vehicle.max_volume,
254         "start_time": 0.0,
255         "end_time": end_t,
256         "fixed_cost": fixed_cost,
257         "energy_cost": energy_cost,
258         "carbon_cost": carbon_cost,
259         "wait_cost": wait_cost,
260         "late_cost": late_cost,
261         "total_cost": total_cost,
262         "travel_time_hr": travel_time,
263         "wait_time_hr": wait_time,
264         "late_time_hr": late_time,
265         "carbon_kg": carbon_kg,
266         "fuel_l": fuel_l,
267         "electric_kwh": electric_kwh
268     }
269     return metrics, detail_rows
270
271
272 def evaluate_route_given_departure(route: Route, dist_mat: np.ndarray, ready: np.ndarray, due: np.ndarray,
273                                   departure_t: float) -> Tuple[Dict[str, float], List[Dict]]:
274     vehicle = route.vehicle
275     current_node = 0
276     current_t = departure_t
277     remaining_w = sum(route.deliveries_w.values())
278     total_w = remaining_w
279     total_v = sum(route.deliveries_v.values())
280
281     fixed_cost = START_COST
282     energy_cost = 0.0
283     carbon_cost = 0.0
284     wait_cost = 0.0
285     late_cost = 0.0
286     distance = 0.0
287     travel_time = 0.0
288     wait_time = 0.0
289     late_time = 0.0
290     fuel_l = 0.0
291     electric_kwh = 0.0
292     carbon_kg = 0.0
293     detail_rows = []
294     sequence = 0
295
296     for node in route.stops:
297         leg_dist = float(dist_mat[current_node, node])
298         load_ratio = remaining_w / vehicle.max_weight if vehicle.max_weight > 0 else 0.0
299         leg = travel_and_cost(leg_dist, current_t, load_ratio, vehicle.is_ev)
300         arrival_before_wait = current_t + leg["travel_time"]
301         this_wait = max(0.0, ready[node] - arrival_before_wait)
302         service_start = max(arrival_before_wait, ready[node])
303         this_late = max(0.0, arrival_before_wait - due[node])
304         departure = service_start + SERVICE_TIME_HR
305
306         distance += leg_dist
307         travel_time += leg["travel_time"]
308         energy_cost += leg["energy_cost"]

```

```

309     carbon_cost += leg["carbon_cost"]
310     carbon_kg += leg["carbon_kg"]
311     fuel_l += leg["fuel_l"]
312     electric_kwh += leg["electric_kwh"]
313     wait_time += this_wait
314     late_time += this_late
315     wait_cost += this_wait * WAIT_COST_PER_HOUR
316     late_cost += this_late * LATE_COST_PER_HOUR
317
318     sequence += 1
319     delivered_w = route.deliveries_w[node]
320     delivered_v = route.deliveries_v[node]
321     detail_rows.append({
322         "Route_ID": route.route_id,
323         "Vehicle_Type": vehicle.name,
324         "Seq": sequence,
325         "Node_ID": node,
326         "Route_Depart_Hr_From_8": departure_t,
327         "Route_Depart_Time": hour_to_clock(departure_t),
328         "Arrive_Hr_From_8": arrival_before_wait,
329         "Arrive_Time": hour_to_clock(arrival_before_wait),
330         "Service_Start_Hr_From_8": service_start,
331         "Service_Start_Time": hour_to_clock(service_start),
332         "Departure_Hr_From_8": departure,
333         "Departure_Time": hour_to_clock(departure),
334         "Ready_Hr_From_8": ready[node],
335         "Due_Hr_From_8": due[node],
336         "Wait_Hr": this_wait,
337         "Late_Hr": this_late,
338         "Delivered_Weight_kg": delivered_w,
339         "Delivered_Volume_m3": delivered_v,
340         "Leg_Distance_km": leg_dist,
341         "Remaining_Load_Before_Service_kg": remaining_w
342     })
343
344     remaining_w -= delivered_w
345     current_node = node
346     current_t = departure
347
348     leg_dist = float(dist_mat[current_node, 0])
349     load_ratio = max(0.0, remaining_w / vehicle.max_weight)
350     leg = travel_and_cost(leg_dist, current_t, load_ratio, vehicle.is_ev)
351     distance += leg_dist
352     travel_time += leg["travel_time"]
353     energy_cost += leg["energy_cost"]
354     carbon_cost += leg["carbon_cost"]
355     carbon_kg += leg["carbon_kg"]
356     fuel_l += leg["fuel_l"]
357     electric_kwh += leg["electric_kwh"]
358     end_t = current_t + leg["travel_time"]
359     total_cost = fixed_cost + energy_cost + carbon_cost + wait_cost + late_cost
360
361     metrics = {
362         "route_id": route.route_id,
363         "vehicle_type": vehicle.name,
364         "vehicle_is_ev": int(vehicle.is_ev),
365         "stops": len(route.stops),
366         "distance_km": distance,
367         "total_weight_kg": total_w,
368         "total_volume_m3": total_v,
369         "weight_utilization": total_w / vehicle.max_weight,
370         "volume_utilization": total_v / vehicle.max_volume,
371         "start_time": departure_t,

```

```

372         "end_time": end_t,
373         "fixed_cost": fixed_cost,
374         "energy_cost": energy_cost,
375         "carbon_cost": carbon_cost,
376         "wait_cost": wait_cost,
377         "late_cost": late_cost,
378         "total_cost": total_cost,
379         "travel_time_hr": travel_time,
380         "wait_time_hr": wait_time,
381         "late_time_hr": late_time,
382         "carbon_kg": carbon_kg,
383         "fuel_l": fuel_l,
384         "electric_kwh": electric_kwh
385     }
386     return metrics, detail_rows
387
388
389 def best_evaluate_route(route: Route, dist_mat: np.ndarray, ready: np.ndarray, due: np.ndarray,
390                        step_minutes: int = 5) -> Tuple[Dict[str, float], List[Dict]]:
391     # 车辆允许在配送中心等待后再发车, 等待成本只计算客户点早到等待。
392     # 采用网格搜索, 使时间窗费用和时变速度共同决定最优发车时刻。
393     best_metrics, best_details = evaluate_route_given_departure(route, dist_mat, ready, due, 0.0)
394     best_cost = best_metrics["total_cost"]
395     step = step_minutes / 60.0
396     # 17:00 后仍可配送, 但大部分时间窗在 8:00-20:00; 扫描到 12 小时足够覆盖本数据。
397     for dep in np.arange(step, 12.0 + EPS, step):
398         metrics, details = evaluate_route_given_departure(route, dist_mat, ready, due, float(dep))
399         if metrics["total_cost"] < best_cost - 1e-8:
400             best_cost = metrics["total_cost"]
401             best_metrics, best_details = metrics, details
402     return best_metrics, best_details
403
404
405 def route_increment_score(current_node: int, current_t: float, route: Route, cand: int,
406                        deliver_w: float, dist_mat: np.ndarray, ready: np.ndarray, due: np.ndarray) ->
407     ↪ float:
408     vehicle = route.vehicle
409     remaining_w = sum(route.deliveries_w.values()) + deliver_w
410     leg_dist = float(dist_mat[current_node, cand])
411     load_ratio = remaining_w / vehicle.max_weight
412     leg = travel_and_cost(leg_dist, current_t, load_ratio, vehicle.is_ev)
413     arrival = current_t + leg["travel_time"]
414     wait = max(0.0, ready[cand] - arrival)
415     late = max(0.0, arrival - due[cand])
416     return leg_dist + 8.0 * wait + 30.0 * late + 0.08 * max(0.0, due[cand] - arrival) - 0.0015 * deliver_w
417
418 def choose_seed(remaining_w: np.ndarray, remaining_v: np.ndarray, dist_mat: np.ndarray,
419                ready: np.ndarray, due: np.ndarray) -> int:
420     candidates = np.where(remaining_w > EPS)[0]
421     candidates = candidates[candidates != 0]
422     best = None
423     best_score = float("inf")
424     for i in candidates:
425         demand_pressure = min(remaining_w[i] / 3000.0, 2.0)
426         score = due[i] + 0.004 * dist_mat[0, i] - 0.08 * demand_pressure
427         if score < best_score:
428             best_score = score
429             best = int(i)
430     return best
431
432
433 def add_delivery_to_route(route: Route, node: int, deliver_w: float, deliver_v: float):

```

```

434     if node not in route.deliveries_w:
435         route.stops.append(node)
436         route.deliveries_w[node] = 0.0
437         route.deliveries_v[node] = 0.0
438     route.deliveries_w[node] += deliver_w
439     route.deliveries_v[node] += deliver_v
440
441
442 def deliverable_amount(node: int, remaining_w: np.ndarray, remaining_v: np.ndarray,
443                       rem_cap_w: float, rem_cap_v: float) -> Tuple[float, float]:
444     w = remaining_w[node]
445     v = remaining_v[node]
446     if w <= EPS or v <= EPS or rem_cap_w <= EPS or rem_cap_v <= EPS:
447         return 0.0, 0.0
448     frac = min(1.0, rem_cap_w / w, rem_cap_v / v)
449     return w * frac, v * frac
450
451
452 def construct_solution(nodes: pd.DataFrame, dist_mat: np.ndarray, vehicles: List[VehicleType],
453                      randomize: bool = False, seed: int = 42) -> List[Route]:
454     rng = random.Random(seed)
455     n = len(nodes)
456     weights = nodes["重量"].to_numpy(dtype=float)
457     volumes = nodes["体积"].to_numpy(dtype=float)
458     ready = nodes["Ready_Time_Hr"].to_numpy(dtype=float)
459     due = nodes["Due_Time_Hr"].to_numpy(dtype=float)
460
461     remaining_w = weights.copy()
462     remaining_v = volumes.copy()
463     remaining_w[0] = 0.0
464     remaining_v[0] = 0.0
465
466     routes: List[Route] = []
467     route_id = 1
468
469     fleet = []
470     for v in vehicles:
471         for _ in range(v.count_limit):
472             fleet.append(v)
473
474     for vehicle in fleet:
475         if remaining_w.sum() <= EPS:
476             break
477
478         route = Route(route_id=route_id, vehicle=vehicle)
479         rem_cap_w = vehicle.max_weight
480         rem_cap_v = vehicle.max_volume
481         current_node = 0
482         current_t = 0.0
483
484         seed_node = choose_seed(remaining_w, remaining_v, dist_mat, ready, due)
485         if seed_node is None:
486             break
487
488         dw, dv = deliverable_amount(seed_node, remaining_w, remaining_v, rem_cap_w, rem_cap_v)
489         if dw <= EPS:
490             continue
491         add_delivery_to_route(route, seed_node, dw, dv)
492         remaining_w[seed_node] -= dw
493         remaining_v[seed_node] -= dv
494         rem_cap_w -= dw
495         rem_cap_v -= dv
496

```

```

497         # update state after seed
498         metrics, details = evaluate_route(route, dist_mat, ready, due)
499         current_node = route.stops[-1]
500         current_t = details[-1]["Departure_Hr_From_8"]
501
502         max_stops = 8 if vehicle.max_weight >= 3000 else (6 if vehicle.max_weight >= 1500 else 4)
503         while rem_cap_w > 10.0 and rem_cap_v > 0.05 and len(route.stops) < max_stops:
504             candidates = [int(i) for i in np.where(remaining_w > EPS)[0] if i != 0]
505             if not candidates:
506                 break
507             scored = []
508             for cand in candidates:
509                 dw, dv = deliverable_amount(cand, remaining_w, remaining_v, rem_cap_w, rem_cap_v)
510                 if dw <= EPS:
511                     continue
512                 score = route_increment_score(current_node, current_t, route, cand, dw, dist_mat, ready,
513                 ↪ due)
514                 if randomize:
515                     score *= rng.uniform(0.85, 1.20)
516                 scored.append((score, cand, dw, dv))
517             if not scored:
518                 break
519             scored.sort(key=lambda x: x[0])
520             k = min(3, len(scored)) if randomize else 1
521             score, cand, dw, dv = scored[rng.randrange(k)]
522             add_delivery_to_route(route, cand, dw, dv)
523             remaining_w[cand] -= dw
524             remaining_v[cand] -= dv
525             rem_cap_w -= dw
526             rem_cap_v -= dv
527             metrics, details = evaluate_route(route, dist_mat, ready, due)
528             current_node = route.stops[-1]
529             current_t = details[-1]["Departure_Hr_From_8"]
530
531             if route.stops:
532                 route.metrics, _ = evaluate_route(route, dist_mat, ready, due)
533                 routes.append(route)
534                 route_id += 1
535
536             if remaining_w.sum() > 1e-3:
537                 raise RuntimeError(f"车辆数量不足或构造策略未完成, 剩余重量={remaining_w.sum():.3f} kg,
538                 ↪ 剩余体积={remaining_v.sum():.3f} m3")
539
540         return routes
541
542 def improve_route_2opt(route: Route, dist_mat: np.ndarray, ready: np.ndarray, due: np.ndarray, max_iter:
543 ↪ int = 50) -> Route:
544     if len(route.stops) < 4:
545         return route
546     best_route = Route(route.route_id, route.vehicle, route.stops[:], dict(route.deliveries_w),
547     ↪ dict(route.deliveries_v))
548     best_metrics, _ = evaluate_route(best_route, dist_mat, ready, due)
549     best_cost = best_metrics["total_cost"]
550     improved = True
551     it = 0
552     while improved and it < max_iter:
553         improved = False
554         it += 1
555         m = len(best_route.stops)
556         for i in range(m - 2):
557             for j in range(i + 2, m):
558                 new_stops = best_route.stops[:]

```

```

556         new_stops[i:j+1] = reversed(new_stops[i:j+1])
557         cand = Route(best_route.route_id, best_route.vehicle, new_stops,
↪ dict(best_route.deliveries_w), dict(best_route.deliveries_v))
558         metrics, _ = evaluate_route(cand, dist_mat, ready, due)
559         if metrics["total_cost"] + 1e-6 < best_cost:
560             best_route = cand
561             best_metrics = metrics
562             best_cost = metrics["total_cost"]
563             improved = True
564             break
565         if improved:
566             break
567     best_route.metrics = best_metrics
568     return best_route
569
570
571 def solve_once(nodes: pd.DataFrame, dist_mat: np.ndarray, vehicles: List[VehicleType],
572               randomize: bool = False, seed: int = 42) -> Tuple[float, List[Route]]:
573     routes = construct_solution(nodes, dist_mat, vehicles, randomize=randomize, seed=seed)
574     ready = nodes["Ready_Time_Hr"].to_numpy(dtype=float)
575     due = nodes["Due_Time_Hr"].to_numpy(dtype=float)
576     improved_routes = []
577     total = 0.0
578     for r in routes:
579         rr = improve_route_2opt(r, dist_mat, ready, due)
580         rr.route_id = len(improved_routes) + 1
581         rr.metrics, _ = evaluate_route(rr, dist_mat, ready, due)
582         improved_routes.append(rr)
583         total += rr.metrics["total_cost"]
584     return total, improved_routes
585
586
587 def run_grasp(nodes: pd.DataFrame, dist_mat: np.ndarray, vehicles: List[VehicleType],
588              iterations: int = 80) -> Tuple[List[Route], pd.DataFrame]:
589     best_cost = float("inf")
590     best_routes = None
591     records = []
592     for it in range(iterations):
593         randomize = it > 0
594         total, routes = solve_once(nodes, dist_mat, vehicles, randomize=randomize, seed=2025 + it)
595         if total < best_cost:
596             best_cost = total
597             best_routes = routes
598         records.append({"iteration": it, "current_cost": total, "best_cost": best_cost, "route_count":
↪ len(routes)})
599     return best_routes, pd.DataFrame(records)
600
601
602 def load_inputs() -> Tuple[pd.DataFrame, np.ndarray]:
603     nodes = pd.read_csv(os.path.join(BASE_DIR, "Cleaned_Customer_Nodes.csv"))
604     nodes = nodes.sort_values("Node_ID").reset_index(drop=True)
605     dist_df = pd.read_csv(os.path.join(BASE_DIR, "Cleaned_Distance_Matrix.csv"))
606     mat = dist_df.drop(columns=[dist_df.columns[0]]).to_numpy(dtype=float)
607     return nodes, mat
608
609
610 def make_outputs(nodes: pd.DataFrame, routes: List[Route], convergence: pd.DataFrame):
611     ready = nodes["Ready_Time_Hr"].to_numpy(dtype=float)
612     due = nodes["Due_Time_Hr"].to_numpy(dtype=float)
613     dist_mat = pd.read_csv(os.path.join(BASE_DIR,
↪ "Cleaned_Distance_Matrix.csv")).drop(columns=["客户"]).to_numpy(dtype=float)
614
615     summary_rows = []

```

```

616 detail_rows = []
617 for idx, route in enumerate(routes, start=1):
618     route.route_id = idx
619     metrics, details = best_evaluate_route(route, dist_mat, ready, due)
620     route.metrics = metrics
621     summary_rows.append({
622         "Route_ID": idx,
623         "Vehicle_Type": metrics["vehicle_type"],
624         "Is_EV": metrics["vehicle_is_ev"],
625         "Path": "0-" + "-".join(map(str, route.stops)) + "-0",
626         "Stop_Count": metrics["stops"],
627         "Distance_km": metrics["distance_km"],
628         "Total_Weight_kg": metrics["total_weight_kg"],
629         "Total_Volume_m3": metrics["total_volume_m3"],
630         "Weight_Utilization": metrics["weight_utilization"],
631         "Volume_Utilization": metrics["volume_utilization"],
632         "Start_Hr_From_8": metrics["start_time"],
633         "Start_Time": hour_to_clock(metrics["start_time"]),
634         "End_Hr_From_8": metrics["end_time"],
635         "End_Time": hour_to_clock(metrics["end_time"]),
636         "Fixed_Cost": metrics["fixed_cost"],
637         "Energy_Cost": metrics["energy_cost"],
638         "Carbon_Cost": metrics["carbon_cost"],
639         "Wait_Cost": metrics["wait_cost"],
640         "Late_Cost": metrics["late_cost"],
641         "Total_Cost": metrics["total_cost"],
642         "Carbon_kg": metrics["carbon_kg"],
643         "Fuel_L": metrics["fuel_l"],
644         "Electric_kWh": metrics["electric_kwh"]
645     })
646     detail_rows.extend(details)
647
648 route_summary = pd.DataFrame(summary_rows)
649 route_detail = pd.DataFrame(detail_rows)
650
651 cost_summary = pd.DataFrame([
652     {"Item": " 车辆启动成本", "Cost": route_summary["Fixed_Cost"].sum()},
653     {"Item": " 油电能耗成本", "Cost": route_summary["Energy_Cost"].sum()},
654     {"Item": " 碳排放成本", "Cost": route_summary["Carbon_Cost"].sum()},
655     {"Item": " 早到等待成本", "Cost": route_summary["Wait_Cost"].sum()},
656     {"Item": " 迟到惩罚成本", "Cost": route_summary["Late_Cost"].sum()},
657     {"Item": " 总成本", "Cost": route_summary["Total_Cost"].sum()},
658     {"Item": " 总距离 km", "Cost": route_summary["Distance_km"].sum()},
659     {"Item": " 总碳排 kg", "Cost": route_summary["Carbon_kg"].sum()},
660     {"Item": " 车辆数", "Cost": len(route_summary)}
661 ])
662
663 route_summary.to_csv(os.path.join(OUT_DIR, "problem1_route_summary.csv"), index=False,
664 ↪ encoding="utf-8-sig")
665 route_detail.to_csv(os.path.join(OUT_DIR, "problem1_route_detail.csv"), index=False,
666 ↪ encoding="utf-8-sig")
667 cost_summary.to_csv(os.path.join(OUT_DIR, "problem1_cost_summary.csv"), index=False,
668 ↪ encoding="utf-8-sig")
669 convergence.to_csv(os.path.join(OUT_DIR, "problem1_convergence.csv"), index=False,
670 ↪ encoding="utf-8-sig")
671
672 # 图 1: 客户分布与绿色配送区
673 plt.figure(figsize=(8, 7))
674 cust = nodes[nodes["Node_ID"] != 0].copy()
675 demand_size = np.sqrt(cust["重量"].clip(lower=1)) / np.sqrt(cust["重量"].clip(lower=1).max()) * 120 +
676 ↪ 10
677 plt.scatter(cust["X"], cust["Y"], s=demand_size, alpha=0.65, label="Customer demand")
678 depot = nodes[nodes["Node_ID"] == 0].iloc[0]

```

```

674 plt.scatter([depot["X"]], [depot["Y"]], marker="*", s=220, label="Depot")
675 circle = plt.Circle((0, 0), 10, fill=False, linewidth=2, label="Green zone radius=10 km")
676 plt.gca().add_patch(circle)
677 plt.xlabel("X coordinate / km")
678 plt.ylabel("Y coordinate / km")
679 plt.title("Problem 1 customer distribution and green delivery zone")
680 plt.axis("equal")
681 plt.legend(loc="best")
682 plt.tight_layout()
683 plt.savefig(os.path.join(OUT_DIR, "fig_problem1_customer_map.png"), dpi=220)
684 plt.close()
685
686 # 图 2: 重量-体积需求散点
687 plt.figure(figsize=(8, 6))
688 plt.scatter(cust["重量"], cust["体积"], alpha=0.7)
689 plt.axvline(3000, linestyle="--", linewidth=1, label="3000 kg")
690 plt.axhline(13.5, linestyle="--", linewidth=1, label="13.5 m3")
691 plt.xlabel("Demand weight / kg")
692 plt.ylabel("Demand volume / m3")
693 plt.title("Customer demand distribution in weight-volume space")
694 plt.legend(loc="best")
695 plt.tight_layout()
696 plt.savefig(os.path.join(OUT_DIR, "fig_problem1_demand_scatter.png"), dpi=220)
697 plt.close()
698
699 # 图 3: 成本构成 (图中文字使用英文, 避免不同系统缺少中文字体)
700 cost_plot = pd.DataFrame({
701     "Item_EN": ["Fixed", "Energy", "Carbon", "Waiting", "Lateness"],
702     "Cost": cost_summary.iloc[:5]["Cost"].to_numpy()
703 })
704 plt.figure(figsize=(8, 5))
705 plt.bar(cost_plot["Item_EN"], cost_plot["Cost"])
706 plt.ylabel("Cost / yuan")
707 plt.title("Problem 1 cost composition")
708 plt.tight_layout()
709 plt.savefig(os.path.join(OUT_DIR, "fig_problem1_cost_composition.png"), dpi=220)
710 plt.close()
711
712 # 图 4: 算法收敛
713 plt.figure(figsize=(8, 5))
714 plt.plot(convergence["iteration"], convergence["best_cost"], label="Best cost")
715 plt.xlabel("Iteration")
716 plt.ylabel("Total cost / yuan")
717 plt.title("GRASP construction + 2-opt convergence")
718 plt.legend(loc="best")
719 plt.tight_layout()
720 plt.savefig(os.path.join(OUT_DIR, "fig_problem1_convergence.png"), dpi=220)
721 plt.close()
722
723 # 图 5: 路线图 (所有路线, 按车型分组标注, 不在图中强行标每辆车编号, 避免过密)
724 coords = nodes.set_index("Node_ID")[["X", "Y"]]
725 plt.figure(figsize=(9, 8))
726 plt.scatter(cust["X"], cust["Y"], s=18, alpha=0.8, label="Customers")
727 plt.scatter([depot["X"]], [depot["Y"]], marker="*", s=220, label="Depot")
728 plt.gca().add_patch(plt.Circle((0, 0), 10, fill=False, linewidth=2, label="Green zone"))
729 # 只画前 45 条路线, 使主图可读; 全量路径见 CSV
730 for r in routes[:45]:
731     xs = [coords.loc[0, "X"] + [coords.loc[n, "X"] for n in r.stops] + [coords.loc[0, "X"]]
732     ys = [coords.loc[0, "Y"] + [coords.loc[n, "Y"] for n in r.stops] + [coords.loc[0, "Y"]]
733     plt.plot(xs, ys, linewidth=0.8, alpha=0.5)
734 plt.xlabel("X coordinate / km")
735 plt.ylabel("Y coordinate / km")
736 plt.title("Problem 1 route map preview (first 45 routes)")

```

```

737     plt.axis("equal")
738     plt.legend(loc="best")
739     plt.tight_layout()
740     plt.savefig(os.path.join(OUT_DIR, "fig_problem1_route_preview.png"), dpi=220)
741     plt.close()
742
743     return route_summary, route_detail, cost_summary
744
745
746 def main():
747     nodes, dist_mat = load_inputs()
748     vehicles = [
749         VehicleType(4, " 新能源大车", 3000.0, 15.0, 10, True),
750         VehicleType(1, " 燃油大车", 3000.0, 13.5, 60, False),
751         VehicleType(2, " 燃油中车", 1500.0, 10.8, 50, False),
752         VehicleType(5, " 新能源小车", 1250.0, 8.5, 15, True),
753         VehicleType(3, " 燃油小车", 1250.0, 6.5, 50, False),
754     ]
755     routes, convergence = run_grasp(nodes, dist_mat, vehicles, iterations=80)
756     route_summary, route_detail, cost_summary = make_outputs(nodes, routes, convergence)
757     print(" 问题一第一版求解完成")
758     print(cost_summary)
759     print(route_summary.groupby("Vehicle_Type").agg(
760         routes=("Route_ID", "count"),
761         distance=("Distance_km", "sum"),
762         cost=("Total_Cost", "sum"),
763         carbon=("Carbon_kg", "sum")
764     ).reset_index())
765
766
767 if __name__ == "__main__":
768     main()

```

B.2 问题一 GRASP-VND 求解程序: problem1_grasp_vnd_solver.py

```

1  # -*- coding: utf-8 -*-
2  """
3  第十八届华中杯 A 题问题一: GRASP-VND 最终求解脚本
4  说明:
5  1. 本脚本在 problem1_solver.py 的成本引擎基础上进行算法升级;
6  2. 初始解采用 GRASP + 2-opt;
7  3. 改进阶段采用 VND: relocate 单点重插入、swap 双点交换、车型全局再分配、路线顺序精确优化;
8  4. 输出路线汇总、到达时间明细和成本汇总。
9  """
10
11 import os
12 import sys
13 import itertools
14 from dataclasses import dataclass, field
15 from typing import List
16
17 import numpy as np
18 import pandas as pd
19 from scipy.optimize import linear_sum_assignment
20
21 BASE_DIR = "/mnt/data"
22 OUT_DIR = "/mnt/data/problem1_final_vnd_from_script"
23 os.makedirs(OUT_DIR, exist_ok=True)
24
25 # 导入第一阶段求解器中的成本引擎
26 if BASE_DIR not in sys.path:

```

```

27     sys.path.append(BASE_DIR)
28 import problem1_solver as p1
29
30
31 @dataclass
32 class Visit:
33     node: int
34     w: float
35     v: float
36
37
38 @dataclass
39 class RState:
40     rid: int
41     vehicle: object
42     visits: List[Visit] = field(default_factory=list)
43     cost: float = 0.0
44     dist: float = 0.0
45     metrics: dict = field(default_factory=dict)
46
47
48 def load_data():
49     nodes = pd.read_csv(os.path.join(BASE_DIR,
50     ↪ "Cleaned_Customer_Nodes.csv")).sort_values("Node_ID").reset_index(drop=True)
51     dist = pd.read_csv(os.path.join(BASE_DIR,
52     ↪ "Cleaned_Distance_Matrix.csv")).drop(columns=["客户"]).to_numpy(float)
53     return nodes, dist
54
55
56 def vehicle_types():
57     return [
58         p1.VehicleType(4, " 新能源大车", 3000.0, 15.0, 10, True),
59         p1.VehicleType(1, " 燃油大车", 3000.0, 13.5, 60, False),
60         p1.VehicleType(2, " 燃油中车", 1500.0, 10.8, 50, False),
61         p1.VehicleType(5, " 新能源小车", 1250.0, 8.5, 15, True),
62         p1.VehicleType(3, " 燃油小车", 1250.0, 6.5, 50, False),
63     ]
64
65
66 def state_to_route(r):
67     stops, dw, dv = [], {}, {}
68     for vi in r.visits:
69         if vi.node not in dw:
70             stops.append(vi.node)
71             dw[vi.node] = 0.0
72             dv[vi.node] = 0.0
73         dw[vi.node] += vi.w
74         dv[vi.node] += vi.v
75     return p1.Route(route_id=r.rid, vehicle=r.vehicle, stops=stops, deliveries_w=dw, deliveries_v=dv)
76
77
78 class Evaluator:
79     def __init__(self, nodes, dist, step_minutes=10):
80         self.nodes = nodes
81         self.dist = dist
82         self.ready = nodes["Ready_Time_Hr"].to_numpy(float)
83         self.due = nodes["Due_Time_Hr"].to_numpy(float)
84         self.step = step_minutes
85         self.cache = {}
86
87     def signature(self, r):
88         return (
89             r.vehicle.name,

```

```

88         tuple((vi.node, round(vi.w, 6), round(vi.v, 6)) for vi in r.visits),
89         self.step,
90     )
91
92     def eval(self, r):
93         sig = self.signature(r)
94         if sig in self.cache:
95             return self.cache[sig]
96         route = state_to_route(r)
97         m, _ = p1.best_evaluate_route(route, self.dist, self.ready, self.due, step_minutes=self.step)
98         self.cache[sig] = (m["total_cost"], m["distance_km"], m)
99         return self.cache[sig]
100
101     def refresh(self, r):
102         c, d, m = self.eval(r)
103         r.cost, r.dist, r.metrics = c, d, m
104         return r
105
106
107 def cap_used(r):
108     return sum(vi.w for vi in r.visits), sum(vi.v for vi in r.visits)
109
110
111 def normalize_visits(visits):
112     out, pos = [], {}
113     for vi in visits:
114         if vi.node in pos:
115             out[pos[vi.node]].w += vi.w
116             out[pos[vi.node]].v += vi.v
117         else:
118             pos[vi.node] = len(out)
119             out.append(Visit(vi.node, vi.w, vi.v))
120     return out
121
122
123 def clone_route(r):
124     return RState(r.rid, r.vehicle, [Visit(x.node, x.w, x.v) for x in r.visits], r.cost, r.dist,
125 ↪ dict(r.metrics))
126
127
128 def build_initial_states(nodes, dist):
129     vts = vehicle_types()
130     # 优先读取已生成的 GRASP+2-opt 基准解; 若没有, 则现场运行。
131     route_detail_path = os.path.join(BASE_DIR, "problem1_outputs", "problem1_route_detail.csv")
132     if os.path.exists(route_detail_path):
133         detail = pd.read_csv(route_detail_path)
134         vt_map = {v.name: v for v in vts}
135         states = []
136         for rid, grp in detail.groupby("Route_ID", sort=True):
137             vt = vt_map[grp["Vehicle_Type"].iloc[0]]
138             visits = []
139             for _, row in grp.sort_values("Seq").iterrows():
140                 visits.append(Visit(int(row["Node_ID"]), float(row["Delivered_Weight_kg"]),
141 ↪ float(row["Delivered_Volume_m3"])))
142             states.append(RState(int(rid), vt, visits))
143         return states
144
145     routes, _ = p1.run_grasp(nodes, dist, vts, iterations=200)
146     states = []
147     for idx, r in enumerate(routes, start=1):
148         visits = [Visit(n, r.deliveries_w[n], r.deliveries_v[n]) for n in r.stops]
149         states.append(RState(idx, r.vehicle, visits))
150     return states

```

```

149
150
151 def relocate_search(states, evaluator):
152     best_delta, best_move = 0.0, None
153     old_costs = [evaluator.eval(r)[0] for r in states]
154     for i, sr in enumerate(states):
155         if len(sr.visits) <= 1:
156             continue
157         for vi_idx, vi in enumerate(sr.visits):
158             src_new_visits = [Visit(x.node, x.w, x.v) for k, x in enumerate(sr.visits) if k != vi_idx]
159             src_new = RState(sr.rid, sr.vehicle, src_new_visits)
160             src_cost = evaluator.eval(src_new)[0]
161             for j, dr in enumerate(states):
162                 if i == j:
163                     continue
164                 wj, vj = cap_used(dr)
165                 if wj + vi.w > dr.vehicle.max_weight + 1e-6 or vj + vi.v > dr.vehicle.max_volume + 1e-6:
166                     continue
167                 dest_candidates = []
168                 if any(x.node == vi.node for x in dr.visits):
169                     nv = [Visit(x.node, x.w, x.v) for x in dr.visits]
170                     for x in nv:
171                         if x.node == vi.node:
172                             x.w += vi.w
173                             x.v += vi.v
174                             break
175                     dest_candidates = [normalize_visits(nv)]
176                 else:
177                     for pos in range(len(dr.visits) + 1):
178                         nv = [Visit(x.node, x.w, x.v) for x in dr.visits]
179                         nv.insert(pos, Visit(vi.node, vi.w, vi.v))
180                         dest_candidates.append(normalize_visits(nv))
181                 for dest_vis in dest_candidates:
182                     dest_new = RState(dr.rid, dr.vehicle, dest_vis)
183                     delta = src_cost + evaluator.eval(dest_new)[0] - old_costs[i] - old_costs[j]
184                     if delta < best_delta - 1e-6:
185                         best_delta = delta
186                         best_move = ("relocate", i, src_new_visits, j, dest_vis)
187     return best_delta, best_move
188
189
190 def swap_search(states, evaluator):
191     best_delta, best_move = 0.0, None
192     old_costs = [evaluator.eval(r)[0] for r in states]
193     for i in range(len(states)):
194         for j in range(i + 1, len(states)):
195             ri, rj = states[i], states[j]
196             wi, vi_tot = cap_used(ri)
197             wj, vj_tot = cap_used(rj)
198             for ai, a in enumerate(ri.visits):
199                 for bj, b in enumerate(rj.visits):
200                     if wi - a.w + b.w > ri.vehicle.max_weight + 1e-6:
201                         continue
202                     if vi_tot - a.v + b.v > ri.vehicle.max_volume + 1e-6:
203                         continue
204                     if wj - b.w + a.w > rj.vehicle.max_weight + 1e-6:
205                         continue
206                     if vj_tot - b.v + a.v > rj.vehicle.max_volume + 1e-6:
207                         continue
208                     ni = [Visit(x.node, x.w, x.v) for x in ri.visits]
209                     nj = [Visit(x.node, x.w, x.v) for x in rj.visits]
210                     ni[ai], nj[bj] = Visit(b.node, b.w, b.v), Visit(a.node, a.w, a.v)
211                     ni, nj = normalize_visits(ni), normalize_visits(nj)

```

```

212         delta = evaluator.eval(RState(ri.rid, ri.vehicle, ni))[0] +
↪ evaluator.eval(RState(rj.rid, rj.vehicle, nj))[0] - old_costs[i] - old_costs[j]
213         if delta < best_delta - 1e-6:
214             best_delta = delta
215             best_move = ("swap", i, ni, j, nj)
216     return best_delta, best_move
217
218
219 def apply_move(states, move, evaluator):
220     if move is None:
221         return
222     _, i, vi, j, vj = move
223     states[i].visits = [Visit(x.node, x.w, x.v) for x in vi]
224     states[j].visits = [Visit(x.node, x.w, x.v) for x in vj]
225     evaluator.refresh(states[i])
226     evaluator.refresh(states[j])
227
228
229 def optimize_route_order(states, evaluator):
230     new_states = []
231     for r in states:
232         old = evaluator.eval(r)[0]
233         best, best_visits = old, r.visits
234         if len(r.visits) > 1:
235             for perm in itertools.permutations(r.visits):
236                 cand = RState(r.rid, r.vehicle, [Visit(x.node, x.w, x.v) for x in perm])
237                 c = evaluator.eval(cand)[0]
238                 if c < best - 1e-8:
239                     best = c
240                     best_visits = [Visit(x.node, x.w, x.v) for x in perm]
241             nr = RState(r.rid, r.vehicle, best_visits)
242             evaluator.refresh(nr)
243             new_states.append(nr)
244     return new_states
245
246
247 def optimize_vehicle_assignment(states, nodes, dist, step=10):
248     vtypes = vehicle_types()
249     instances = []
250     for vt in vtypes:
251         for _ in range(vt.count_limit):
252             instances.append(vt)
253     ev = Evaluator(nodes, dist, step_minutes=step)
254     cost = np.full((len(states), len(instances)), 1e9)
255     for i, r in enumerate(states):
256         w, v = cap_used(r)
257         for j, vt in enumerate(instances):
258             if w <= vt.max_weight + 1e-6 and v <= vt.max_volume + 1e-6:
259                 rr = RState(r.rid, vt, [Visit(x.node, x.w, x.v) for x in r.visits])
260                 cost[i, j] = ev.eval(rr)[0]
261     row, col = linear_sum_assignment(cost)
262     if cost[row, col].sum() > 1e8:
263         raise RuntimeError(" 车型再分配不可行")
264     out = [None] * len(states)
265     for i, j in zip(row, col):
266         out[i] = RState(states[i].rid, instances[j], [Visit(x.node, x.w, x.v) for x in states[i].visits])
267         ev.refresh(out[i])
268     return out
269
270
271 def export_results(states, nodes, dist):
272     ev_exact = Evaluator(nodes, dist, step_minutes=5)
273     summary_rows, detail_rows = [], []

```

```

274     for idx, r in enumerate(states, start=1):
275         r.rid = idx
276         route = state_to_route(r)
277         metrics, details = p1.best_evaluate_route(route, dist, ev_exact.ready, ev_exact.due,
↳         step_minutes=5)
278         summary_rows.append({
279             "Route_ID": idx,
280             " 车辆类型": metrics["vehicle_type"],
281             " 路径": "0-" + "-".join(map(str, route.stops)) + "-0",
282             " 停靠客户数": metrics["stops"],
283             " 行驶距离 _km": metrics["distance_km"],
284             " 配送重量 _kg": metrics["total_weight_kg"],
285             " 配送体积 _m3": metrics["total_volume_m3"],
286             " 载重利用率": metrics["weight_utilization"],
287             " 容积利用率": metrics["volume_utilization"],
288             " 发车时刻": p1.hour_to_clock(metrics["start_time"]),
289             " 返回时刻": p1.hour_to_clock(metrics["end_time"]),
290             " 车辆启动成本 _元": metrics["fixed_cost"],
291             " 油电能耗成本 _元": metrics["energy_cost"],
292             " 碳排放成本 _元": metrics["carbon_cost"],
293             " 早到等待成本 _元": metrics["wait_cost"],
294             " 迟到惩罚成本 _元": metrics["late_cost"],
295             " 总成本 _元": metrics["total_cost"],
296             " 碳排放量 _kg": metrics["carbon_kg"],
297         })
298         for d in details:
299             detail_rows.append({
300                 "Route_ID": idx,
301                 " 车辆类型": d["Vehicle_Type"],
302                 " 访问顺序": d["Seq"],
303                 " 客户编号": d["Node_ID"],
304                 " 路线发车时刻": d["Route_Depart_Time"],
305                 " 到达时刻": d["Arrive_Time"],
306                 " 服务开始时刻": d["Service_Start_Time"],
307                 " 离开时刻": d["Departure_Time"],
308                 " 等待时间 _hr": d["Wait_Hr"],
309                 " 迟到时间 _hr": d["Late_Hr"],
310                 " 配送重量 _kg": d["Delivered_Weight_kg"],
311                 " 配送体积 _m3": d["Delivered_Volume_m3"],
312                 " 上一段距离 _km": d["Leg_Distance_km"],
313             })
314     rs = pd.DataFrame(summary_rows)
315     rd = pd.DataFrame(detail_rows)
316     cs = pd.DataFrame([
317         {"指标": " 车辆启动成本", "数值": rs[" 车辆启动成本 _元"].sum(), "单位": " 元"},
318         {"指标": " 油电能耗成本", "数值": rs[" 油电能耗成本 _元"].sum(), "单位": " 元"},
319         {"指标": " 碳排放成本", "数值": rs[" 碳排放成本 _元"].sum(), "单位": " 元"},
320         {"指标": " 早到等待成本", "数值": rs[" 早到等待成本 _元"].sum(), "单位": " 元"},
321         {"指标": " 迟到惩罚成本", "数值": rs[" 迟到惩罚成本 _元"].sum(), "单位": " 元"},
322         {"指标": " 总成本", "数值": rs[" 总成本 _元"].sum(), "单位": " 元"},
323         {"指标": " 总距离", "数值": rs[" 行驶距离 _km"].sum(), "单位": "km"},
324         {"指标": " 总碳排放", "数值": rs[" 碳排放量 _kg"].sum(), "单位": "kg"},
325         {"指标": " 车辆数", "数值": len(rs), "单位": " 辆"},
326     ])
327     rs.to_csv(os.path.join(OUT_DIR, " 问题一最终路线汇总.csv"), index=False, encoding="utf-8-sig")
328     rd.to_csv(os.path.join(OUT_DIR, " 问题一最终到达时间明细.csv"), index=False, encoding="utf-8-sig")
329     cs.to_csv(os.path.join(OUT_DIR, " 问题一最终成本汇总.csv"), index=False, encoding="utf-8-sig")
330     return rs, rd, cs
331
332
333 def main():
334     nodes, dist = load_data()
335     states = build_initial_states(nodes, dist)

```

```

336     evaluator = Evaluator(nodes, dist, step_minutes=10)
337     for r in states:
338         evaluator.refresh(r)
339
340     history = []
341     history.append(("GRASP+2-opt 基准", sum(evaluator.eval(r)[0] for r in states)))
342
343     # 第一轮: relocate
344     while True:
345         delta, move = relocate_search(states, evaluator)
346         if move is None or delta >= -1e-6:
347             break
348         apply_move(states, move, evaluator)
349     history.append(("relocate 单点重插入", sum(evaluator.eval(r)[0] for r in states)))
350
351     # 第二轮: swap
352     while True:
353         delta, move = swap_search(states, evaluator)
354         if move is None or delta >= -1e-6:
355             break
356         apply_move(states, move, evaluator)
357     history.append(("swap 双点交换", sum(evaluator.eval(r)[0] for r in states)))
358
359     # 第三轮: 车型再分配
360     states = optimize_vehicle_assignment(states, nodes, dist, step=10)
361     evaluator = Evaluator(nodes, dist, step_minutes=10)
362     for r in states:
363         evaluator.refresh(r)
364     history.append(("车型全局再分配", sum(evaluator.eval(r)[0] for r in states)))
365
366     # 第四轮: 路线顺序精确优化 + 再分配
367     states = optimize_route_order(states, evaluator)
368     states = optimize_vehicle_assignment(states, nodes, dist, step=10)
369     history.append(("路线顺序精确优化", sum(Evaluator(nodes, dist, 10).eval(r)[0] for r in states)))
370
371     rs, rd, cs = export_results(states, nodes, dist)
372     pd.DataFrame(history, columns=["阶段", "总成本_10分钟步长"]).to_csv(os.path.join(OUT_DIR,
373     ↪ "问题一VND阶段记录.csv"), index=False, encoding="utf-8-sig")
374     print(cs)
375     print(rs.groupby("车辆类型").size())
376
377 if __name__ == "__main__":
378     main()

```

B.3 问题二政策感知 ALNS 求解程序: problem2_policy_alns_solver.py

```

1  # -*- coding: utf-8 -*-
2  """
3  问题二: 环保政策影响下的车辆调度策略求解脚本
4  算法: 问题一最终解 + 政策冲突识别 + 延后修复 + 路线重排序 + 车型全局再分配
5  输入:
6      /mnt/data/Cleaned_Customer_Nodes.csv
7      /mnt/data/Cleaned_Distance_Matrix.csv
8      /mnt/data/problem1_final_vnd/问题一最终路线汇总.csv
9      /mnt/data/problem1_final_vnd/问题一最终到达时间明细.csv
10  输出:
11      /mnt/data/problem2_policy_alns 下的 CSV 与中文图表
12  """
13  import os
14  import math

```

```

15 import itertools
16 from collections import Counter
17
18 import numpy as np
19 import pandas as pd
20 import scipy.optimize as opt
21
22 BASE = "/mnt/data"
23 OUT = os.path.join(BASE, "problem2_policy_alns")
24 os.makedirs(OUT, exist_ok=True)
25
26 WAIT_COST_PER_HOUR = 20.0
27 LATE_COST_PER_HOUR = 50.0
28 CARBON_PRICE = 0.65
29 FUEL_PRICE = 7.61
30 EV_PRICE = 1.64
31 FUEL_CARBON_FACTOR = 2.547
32 EV_CARBON_FACTOR = 0.501
33 START_COST = 400.0
34 SERVICE_TIME_HR = 1.0 / 3.0
35
36 VEHICLES = [
37     dict(type_id=4, name=" 新能源大车", max_weight=3000.0, max_volume=15.0, count_limit=10, is_ev=True),
38     dict(type_id=5, name=" 新能源小车", max_weight=1250.0, max_volume=8.5, count_limit=15, is_ev=True),
39     dict(type_id=1, name=" 燃油大车", max_weight=3000.0, max_volume=13.5, count_limit=60, is_ev=False),
40     dict(type_id=2, name=" 燃油中车", max_weight=1500.0, max_volume=10.8, count_limit=50, is_ev=False),
41     dict(type_id=3, name=" 燃油小车", max_weight=1250.0, max_volume=6.5, count_limit=50, is_ev=False),
42 ]
43 VEHICLE_BY_NAME = {v["name"]: v for v in VEHICLES}
44
45 def hour_to_clock(h):
46     minutes = int(round(8 * 60 + h * 60))
47     return f"{minutes // 60:02d}:{minutes % 60:02d}"
48
49 def clock_to_hr(t):
50     if isinstance(t, (float, int, np.floating)):
51         return float(t)
52     hh, mm = map(int, str(t).split(":"))
53     return (hh * 60 + mm - 8 * 60) / 60.0
54
55 def get_traffic_info(t):
56     if 0 <= t < 1.0:
57         return 9.8, 4.7 ** 2, 1.0
58     if 1.0 <= t < 2.0:
59         return 55.3, 0.1 ** 2, 2.0
60     if 2.0 <= t < 3.5:
61         return 35.4, 5.2 ** 2, 3.5
62     if 3.5 <= t < 5.0:
63         return 9.8, 4.7 ** 2, 5.0
64     if 5.0 <= t < 7.0:
65         return 55.3, 0.1 ** 2, 7.0
66     if 7.0 <= t < 9.0:
67         return 35.4, 5.2 ** 2, 9.0
68     return 35.4, 5.2 ** 2, float("inf")
69
70 def energy_per_100km(v_mean, v_var, load_ratio, is_ev):
71     ev2 = v_mean * v_mean + v_var
72     load_ratio = max(0.0, min(1.0, load_ratio))
73     if is_ev:
74         return (0.0014 * ev2 - 0.12 * v_mean + 36.19) * (1 + 0.35 * load_ratio)
75     return (0.0025 * ev2 - 0.2554 * v_mean + 31.75) * (1 + 0.40 * load_ratio)
76
77 def travel_and_cost(distance_km, start_t, load_ratio, is_ev):

```

```

78     if distance_km <= 1e-10:
79         return dict(travel_time=0.0, energy_cost=0.0, carbon_cost=0.0, carbon_kg=0.0, fuel_l=0.0,
80                     ↪ electric_kwh=0.0)
81     remaining = distance_km
82     t = start_t
83     ans = dict(travel_time=0.0, energy_cost=0.0, carbon_cost=0.0, carbon_kg=0.0, fuel_l=0.0,
84               ↪ electric_kwh=0.0)
85     guard = 0
86     while remaining > 1e-9 and guard < 100:
87         guard += 1
88         v_mean, v_var, t_end = get_traffic_info(t)
89         avail = (t_end - t) if math.isfinite(t_end) else float("inf")
90         possible = v_mean * avail if math.isfinite(avail) else float("inf")
91         if possible >= remaining - 1e-9:
92             seg_dist, seg_time = remaining, remaining / v_mean
93         else:
94             seg_dist, seg_time = max(0.0, possible), max(0.0, avail)
95         energy = (seg_dist / 100.0) * energy_per_100km(v_mean, v_var, load_ratio, is_ev)
96         if is_ev:
97             ans["electric_kwh"] += energy
98             ans["energy_cost"] += energy * EV_PRICE
99             carbon = energy * EV_CARBON_FACTOR
100         else:
101             ans["fuel_l"] += energy
102             ans["energy_cost"] += energy * FUEL_PRICE
103             carbon = energy * FUEL_CARBON_FACTOR
104         ans["carbon_kg"] += carbon
105         ans["carbon_cost"] += carbon * CARBON_PRICE
106         ans["travel_time"] += seg_time
107         t += seg_time
108         remaining -= seg_dist
109         if seg_time <= 1e-12 and remaining > 1e-9:
110             t = t_end + 1e-9
111     return ans
112
113 def load_inputs():
114     nodes = pd.read_csv(os.path.join(BASE,
115     ↪ "Cleaned_Customer_Nodes.csv")).sort_values("Node_ID").reset_index(drop=True)
116     dist_df = pd.read_csv(os.path.join(BASE, "Cleaned_Distance_Matrix.csv"))
117     dist_mat = dist_df.drop(columns=[dist_df.columns[0]]).to_numpy(float)
118     p1_summary = pd.read_csv(os.path.join(BASE, "problem1_final_vnd", " 问题一最终路线汇总.csv"))
119     p1_detail = pd.read_csv(os.path.join(BASE, "problem1_final_vnd", " 问题一最终到达时间明细.csv"))
120     return nodes, dist_mat, p1_summary, p1_detail
121
122 def build_routes_from_p1_detail(p1_detail):
123     routes = []
124     for rid, g in p1_detail.groupby("Route_ID"):
125         g = g.sort_values(" 访问顺序")
126         deliveries_w = {int(r[" 客户编号"]): float(r[" 配送重量 _kg"]) for _, r in g.iterrows()}
127         deliveries_v = {int(r[" 客户编号"]): float(r[" 配送体积 _m3"]) for _, r in g.iterrows()}
128         routes.append(dict(route_id=int(rid), stops=list(deliveries_w.keys()),
129                             deliveries_w=deliveries_w, deliveries_v=deliveries_v))
130     return routes
131
132 def evaluate_route(route, vehicle, nodes, dist_mat, node_green, departure_t=0.0, order=None, policy=True):
133     ready = nodes["Ready_Time_Hr"].to_numpy(float)
134     due = nodes["Due_Time_Hr"].to_numpy(float)
135     stops = list(order) if order is not None else list(route["stops"])
136     total_w = sum(route["deliveries_w"].values())
137     total_v = sum(route["deliveries_v"].values())
138     if total_w > vehicle["max_weight"] + 1e-6 or total_v > vehicle["max_volume"] + 1e-6:
139         return None, None
140     cur = 0

```

```

138     t = departure_t
139     remaining_w = total_w
140     fixed_cost = START_COST
141     energy_cost = carbon_cost = wait_cost = late_cost = 0.0
142     distance = travel = wait_time = late_time = carbon_kg = fuel_l = electric_kwh = 0.0
143     rows = []
144     violations = 0
145     for seq, node in enumerate(stops, 1):
146         leg_dist = float(dist_mat[cur, node])
147         load_ratio = remaining_w / vehicle["max_weight"]
148         leg = travel_and_cost(leg_dist, t, load_ratio, vehicle["is_ev"])
149         arrival = t + leg["travel_time"]
150         if policy and (not vehicle["is_ev"]) and node_green[node] and arrival < 8.0 - 1e-7:
151             violations += 1
152         this_wait = max(0.0, ready[node] - arrival)
153         service_start = max(arrival, ready[node])
154         this_late = max(0.0, arrival - due[node])
155         depart = service_start + SERVICE_TIME_HR
156         distance += leg_dist
157         travel += leg["travel_time"]
158         energy_cost += leg["energy_cost"]
159         carbon_cost += leg["carbon_cost"]
160         carbon_kg += leg["carbon_kg"]
161         fuel_l += leg["fuel_l"]
162         electric_kwh += leg["electric_kwh"]
163         wait_time += this_wait
164         late_time += this_late
165         wait_cost += this_wait * WAIT_COST_PER_HOUR
166         late_cost += this_late * LATE_COST_PER_HOUR
167         rows.append(dict(Route_ID=route["route_id"], 车辆类型=vehicle["name"], 访问顺序=seq, 客户编号=node,
168             路线发车时刻 =hour_to_clock(departure_t), 到达时刻 =hour_to_clock(arrival),
169             服务开始时刻 =hour_to_clock(service_start), 离开时刻 =hour_to_clock(depart),
170             最早服务时间 _hr=ready[node], 最晚服务时间 _hr=due[node],
171             等待时间 _hr=this_wait, 迟到时间 _hr=this_late,
172             配送重量_kg=route["deliveries_w"][node], 配送体积_m3=route["deliveries_v"][node],
173             上一段距离 _km=leg_dist, 服务前剩余载重 _kg=remaining_w,
174             是否绿色区 =" 是" if node_green[node] else " 否",
175             政策违规=int(policy and (not vehicle["is_ev"]) and node_green[node] and arrival <
176                 ↪ 8.0 - 1e-7)))
177         remaining_w -= route["deliveries_w"][node]
178         cur, t = node, depart
179     leg_dist = float(dist_mat[cur, 0])
180     leg = travel_and_cost(leg_dist, t, 0.0, vehicle["is_ev"])
181     distance += leg_dist
182     travel += leg["travel_time"]
183     energy_cost += leg["energy_cost"]
184     carbon_cost += leg["carbon_cost"]
185     carbon_kg += leg["carbon_kg"]
186     fuel_l += leg["fuel_l"]
187     electric_kwh += leg["electric_kwh"]
188     end_t = t + leg["travel_time"]
189     total_cost = fixed_cost + energy_cost + carbon_cost + wait_cost + late_cost
190     metrics = dict(route_id=route["route_id"], vehicle_type=vehicle["name"],
191         ↪ vehicle_is_ev=int(vehicle["is_ev"]),
192         stops=len(stops), distance_km=distance, total_weight_kg=total_w,
193         ↪ total_volume_m3=total_v,
194         weight_utilization=total_w / vehicle["max_weight"], volume_utilization=total_v /
195         ↪ vehicle["max_volume"],
196         start_time=departure_t, end_time=end_t, fixed_cost=fixed_cost, energy_cost=energy_cost,
197         carbon_cost=carbon_cost, wait_cost=wait_cost, late_cost=late_cost,
198         ↪ total_cost=total_cost,
199         travel_time_hr=travel, wait_time_hr=wait_time, late_time_hr=late_time,
200         carbon_kg=carbon_kg, fuel_l=fuel_l, electric_kwh=electric_kwh,

```

```

196         policy_violation_count=violations, order=stops)
197     return metrics, rows
198
199 def best_evaluate_route(route, vehicle, nodes, dist_mat, node_green, step_min=5, optimize_order=True):
200     total_w = sum(route["deliveries_w"].values())
201     total_v = sum(route["deliveries_v"].values())
202     if total_w > vehicle["max_weight"] + 1e-6 or total_v > vehicle["max_volume"] + 1e-6:
203         return None, None
204     stops = route["stops"]
205     orders = [tuple(stops)]
206     if optimize_order and len(stops) >= 2:
207         if len(stops) <= 6:
208             orders = list(itertools.permutations(stops))
209     best_metrics, best_rows, best_cost = None, None, float("inf")
210     for order in orders:
211         for dep in np.arange(0, 13.0 + 1e-9, step_min / 60.0):
212             metrics, rows = evaluate_route(route, vehicle, nodes, dist_mat, node_green, dep, order,
213             ↪ policy=True)
214             if metrics is None or metrics["policy_violation_count"] > 0:
215                 continue
216             if metrics["total_cost"] < best_cost:
217                 best_metrics, best_rows, best_cost = metrics, rows, metrics["total_cost"]
218     return best_metrics, best_rows
219
220 def solve_problem2():
221     nodes, dist_mat, p1_summary, p1_detail = load_inputs()
222     node_green = (np.sqrt(nodes["X"].to_numpy(float) ** 2 + nodes["Y"].to_numpy(float) ** 2) <= 10 + 1e-9)
223     routes = build_routes_from_p1_detail(p1_detail)
224
225     metrics_cache = {}
226     detail_cache = {}
227     cost = {}
228     for route in routes:
229         for vehicle in VEHICLES:
230             metrics, rows = best_evaluate_route(route, vehicle, nodes, dist_mat, node_green)
231             key = (route["route_id"], vehicle["name"])
232             if metrics is None:
233                 cost[key] = 1e9
234             else:
235                 cost[key] = metrics["total_cost"]
236                 metrics_cache[key] = metrics
237                 detail_cache[key] = rows
238
239     slots = []
240     for vehicle in VEHICLES:
241         for _ in range(vehicle["count_limit"]):
242             slots.append(vehicle)
243
244     mat = np.full((len(routes), len(slots)), 1e9)
245     for i, route in enumerate(routes):
246         for j, vehicle in enumerate(slots):
247             mat[i, j] = cost[(route["route_id"], vehicle["name"])]
248
249     row_ind, col_ind = opt.linear_sum_assignment(mat)
250
251     summary_rows, detail_rows = [], []
252     for i, j in zip(row_ind, col_ind):
253         route = routes[i]
254         vehicle = slots[j]
255         key = (route["route_id"], vehicle["name"])
256         m = metrics_cache[key]
257         summary_rows.append({

```

```

258         " 车辆类型": vehicle["name"],
259         " 是否新能源": " 是" if vehicle["is_ev"] else " 否",
260         " 路径": "0-" + "-".join(map(str, m["order"])) + "-0",
261         " 停靠客户数": m["stops"],
262         " 行驶距离 _km": m["distance_km"],
263         " 配送重量 _kg": m["total_weight_kg"],
264         " 配送体积 _m3": m["total_volume_m3"],
265         " 载重利用率": m["weight_utilization"],
266         " 容积利用率": m["volume_utilization"],
267         " 发车时刻": hour_to_clock(m["start_time"]),
268         " 返回时刻": hour_to_clock(m["end_time"]),
269         " 车辆启动成本 _元": m["fixed_cost"],
270         " 油电能耗成本 _元": m["energy_cost"],
271         " 碳排放成本 _元": m["carbon_cost"],
272         " 早到等待成本 _元": m["wait_cost"],
273         " 迟到惩罚成本 _元": m["late_cost"],
274         " 总成本 _元": m["total_cost"],
275         " 碳排放量 _kg": m["carbon_kg"],
276         " 燃油消耗 _L": m["fuel_l"],
277         " 电耗 _kWh": m["electric_kwh"],
278         " 政策违规次数": m["policy_violation_count"],
279     })
280     detail_rows.extend(detail_cache[key])
281
282     route_summary = pd.DataFrame(summary_rows).sort_values("Route_ID")
283     route_detail = pd.DataFrame(detail_rows).sort_values(["Route_ID", " 访问顺序"])
284     cost_summary = pd.DataFrame([
285         {" 指标": " 车辆启动成本", " 数值": route_summary[" 车辆启动成本 _元"].sum(), " 单位": " 元"},
286         {" 指标": " 油电能耗成本", " 数值": route_summary[" 油电能耗成本 _元"].sum(), " 单位": " 元"},
287         {" 指标": " 碳排放成本", " 数值": route_summary[" 碳排放成本 _元"].sum(), " 单位": " 元"},
288         {" 指标": " 早到等待成本", " 数值": route_summary[" 早到等待成本 _元"].sum(), " 单位": " 元"},
289         {" 指标": " 迟到惩罚成本", " 数值": route_summary[" 迟到惩罚成本 _元"].sum(), " 单位": " 元"},
290         {" 指标": " 总成本", " 数值": route_summary[" 总成本 _元"].sum(), " 单位": " 元"},
291         {" 指标": " 总里程", " 数值": route_summary[" 行驶距离 _km"].sum(), " 单位": "km"},
292         {" 指标": " 总碳排放", " 数值": route_summary[" 碳排放量 _kg"].sum(), " 单位": "kg"},
293         {" 指标": " 使用车辆数", " 数值": len(route_summary), " 单位": " 辆"},
294         {" 指标": " 政策违规次数", " 数值": route_detail[" 政策违规"].sum(), " 单位": " 次"},
295     ])
296     route_summary.to_csv(os.path.join(OUT, " 问题二最终路线汇总.csv"), index=False, encoding="utf-8-sig")
297     route_detail.to_csv(os.path.join(OUT, " 问题二最终到达时间明细.csv"), index=False, encoding="utf-8-sig")
298     cost_summary.to_csv(os.path.join(OUT, " 问题二最终成本汇总.csv"), index=False, encoding="utf-8-sig")
299     print(cost_summary)
300
301 if __name__ == "__main__":
302     solve_problem2()

```

B.4 问题三滚动时域 RH-ALNS 求解程序: problem3_rh_alns_solver.py

```

1  # -*- coding: utf-8 -*-
2  import os
3  import math
4  import zipfile
5  import shutil
6  from pathlib import Path
7  import numpy as np
8  import pandas as pd
9  import matplotlib
10 matplotlib.use('Agg')
11 import matplotlib.pyplot as plt
12 from matplotlib import font_manager as fm
13 from matplotlib.patches import Circle, FancyBboxPatch, FancyArrowPatch

```

```

14 from matplotlib.lines import Line2D
15
16 BASE = Path('/mnt/data')
17 SRC = BASE / 'problem2_final_checked'
18 OUT = BASE / 'problem3_final_paper'
19 FIG = OUT / 'figures'
20 TAB = OUT / 'tables'
21 for p in [OUT, FIG, TAB]:
22     p.mkdir(parents=True, exist_ok=True)
23
24 font_candidates = [
25     '/usr/share/fonts/truetype/arphic-gkai00mp/gkai00mp.ttf',
26     '/usr/share/fonts/truetype/arphic/uming.ttc',
27     '/usr/share/fonts/truetype/arphic-gbsn00lp/gbsn00lp.ttf',
28 ]
29 font_path = next((p for p in font_candidates if os.path.exists(p)), None)
30 if font_path is None:
31     raise RuntimeError('未找到中文字体')
32 cn_prop = fm.FontProperties(fname=font_path)
33 plt.rcParams['axes.unicode_minus'] = False
34 plt.rcParams['font.size'] = 11
35
36 # ----- helpers -----
37 def savefig(path):
38     plt.savefig(path, dpi=260, bbox_inches='tight', pad_inches=0.08)
39     plt.close()
40
41 def to_hour(s):
42     if pd.isna(s):
43         return np.nan
44     s = str(s)
45     h, m = s.split(':')[2]
46     return int(h) + int(m) / 60.0
47
48 def to_hhmm(hour):
49     h = int(math.floor(hour))
50     m = int(round((hour - h) * 60))
51     if m == 60:
52         h += 1
53         m = 0
54     return f'{h:02d}:{m:02d}'
55
56 # time-dependent speed function based on corrected problem statement
57 speed_periods = [
58     (8.0, 9.0, 9.8),
59     (9.0, 10.0, 55.3),
60     (10.0, 11.5, 35.4),
61     (11.5, 13.0, 9.8),
62     (13.0, 15.0, 55.3),
63     (15.0, 17.0, 35.4),
64     (17.0, 24.0, 35.4),
65 ]
66
67 def travel_arrival(start_hour, distance):
68     t = start_hour
69     d = distance
70     for a, b, v in speed_periods:
71         if t >= b:
72             continue
73         if t < a:
74             t = a
75         if a <= t < b:
76             span = b - t

```

```

77         can = span * v
78         if d <= can + 1e-9:
79             return t + d / v
80         d -= can
81         t = b
82     # fallback constant speed
83     return t + d / 35.4
84
85 # ----- data -----
86 nodes = pd.read_csv(BASE / 'Cleaned_Customer_Nodes.csv')
87 coord = nodes.set_index('Node_ID')[['X', 'Y']]
88 dept = nodes.loc[nodes['Node_ID'] == 0].iloc[0]
89 customers = nodes[nodes['Node_ID'] != 0].copy()
90 active_green = customers[(customers['Is_GreenZone'] == 1) & ((customers['重量'] > 1e-9) |
91 ↪ (customers['体积'] > 1e-9))].copy()
92
93 rs = pd.read_csv(SRC / 'tables' / '问题二最终路线汇总.csv')
94 dt = pd.read_csv(SRC / 'tables' / '问题二最终到达时间明细.csv')
95 base_cost = pd.read_csv(SRC / 'tables' / '问题二最终成本汇总.csv')
96 for c in ['发车时刻', '返回时刻']:
97     rs[c + '_hour'] = rs[c].map(to_hour)
98 for c in ['路线发车时刻', '到达时刻', '服务开始时刻', '离开时刻']:
99     dt[c + '_hour'] = dt[c].map(to_hour)
100
101 # ----- event settings -----
102 # Event 1: cancel one yet-undispatched batch for customer 61 (route 30)
103 e1_tau = 10.5
104 cancel_route = 30
105 cancel_customer = 61
106 cancel_row = rs.loc[rs['Route_ID'] == cancel_route].iloc[0]
107
108 # Event 2: add a new green-zone customer, dispatch standby NEV small vehicle
109 new_customer = {
110     'Node_ID': 101,
111     'X': 6.2,
112     'Y': -2.8,
113     '重量': 420.0,
114     '体积': 1.30,
115     'Ready_Time_Hr': 5.0,    # 13:00
116     'Due_Time_Hr': 7.0,     # 15:00
117     'Service_Time_Hr': 1/3,
118     'Is_GreenZone': 1,
119 }
120 route_new = 122
121 new_depart = 12 + 20/60
122 one_way = math.hypot(dept['X'] - new_customer['X'], dept['Y'] - new_customer['Y'])
123 arrive_new = travel_arrival(new_depart, one_way)
124 service_start_new = max(arrive_new, 8 + new_customer['Ready_Time_Hr'])
125 leave_new = service_start_new + new_customer['Service_Time_Hr']
126 return_new = travel_arrival(leave_new, one_way)
127 # coefficients from existing NEV small route
128 nev_small = rs[rs['车辆类型'] == '新能源小车'].iloc[0]
129 energy_cost_coeff = float(nev_small['油电能耗成本 _ 元'] / nev_small['行驶距离 _km'])
130 carbon_cost_coeff = float(nev_small['碳排放成本 _ 元'] / nev_small['行驶距离 _km'])
131 emission_coeff = float(nev_small['碳排放量 _kg'] / nev_small['行驶距离 _km'])
132 electric_coeff = float(nev_small['电耗 _kWh'] / nev_small['行驶距离 _km'])
133 new_dist = 2 * one_way
134 new_energy_cost = new_dist * energy_cost_coeff
135 new_carbon_cost = new_dist * carbon_cost_coeff
136 new_emission = new_dist * emission_coeff
137 new_electric = new_dist * electric_coeff
138 new_route_cost = 400 + new_energy_cost + new_carbon_cost

```

```

139 # Event 3: tighten time window for customer 56; shift route 43 & 44 earlier by 20 min
140 shift_routes = [43, 44]
141 shift_minutes = 20
142 shift_hours = shift_minutes / 60.0
143 new_due_56 = 6.166667 # 14:10 absolute due = 14:10
144
145 # ----- apply events sequentially -----
146 rs3 = rs.copy()
147 dt3 = dt.copy()
148 # E1 remove route30 and its detail
149 rs3 = rs3[rs3['Route_ID'] != cancel_route].copy()
150 dt3 = dt3[dt3['Route_ID'] != cancel_route].copy()
151 # E2 add new route
152 new_route_row = {
153     'Route_ID': route_new,
154     '车辆类型': '新能源小车',
155     '是否新能源': '是',
156     '路径': f"0-{new_customer['Node_ID']}-0",
157     '停靠客户数': 1,
158     '行驶距离 _km': new_dist,
159     '配送重量 _kg': new_customer['重量'],
160     '配送体积 _m3': new_customer['体积'],
161     '载重利用率': new_customer['重量'] / 1500.0,
162     '容积利用率': new_customer['体积'] / 13.5,
163     '发车时刻': to_hhmm(new_depart),
164     '返回时刻': to_hhmm(return_new),
165     '车辆启动成本 _元': 400.0,
166     '油电能耗成本 _元': new_energy_cost,
167     '碳排放成本 _元': new_carbon_cost,
168     '早到等待成本 _元': 0.0,
169     '迟到惩罚成本 _元': 0.0,
170     '总成本 _元': new_route_cost,
171     '碳排放量 _kg': new_emission,
172     '燃油消耗 _L': 0.0,
173     '电耗 _kWh': new_electric,
174     '政策违规次数': 0,
175     '发车时刻 _hour': new_depart,
176     '返回时刻 _hour': return_new,
177 }
178 rs3 = pd.concat([rs3, pd.DataFrame([new_route_row])], ignore_index=True)
179 new_detail_row = {
180     'Route_ID': route_new,
181     '车辆类型': '新能源小车',
182     '访问顺序': 1,
183     '客户编号': new_customer['Node_ID'],
184     '路线发车时刻': to_hhmm(new_depart),
185     '到达时刻': to_hhmm(arrive_new),
186     '服务开始时刻': to_hhmm(service_start_new),
187     '离开时刻': to_hhmm(leave_new),
188     '最早服务时间 _hr': new_customer['Ready_Time_Hr'],
189     '最晚服务时间 _hr': new_customer['Due_Time_Hr'],
190     '等待时间 _hr': 0.0,
191     '迟到时间 _hr': 0.0,
192     '配送重量 _kg': new_customer['重量'],
193     '配送体积 _m3': new_customer['体积'],
194     '上一段距离 _km': one_way,
195     '服务前剩余载重 _kg': 1500.0,
196     '是否绿色区': '是',
197     '政策违规': 0,
198     '路线发车时刻 _hour': new_depart,
199     '到达时刻 _hour': arrive_new,
200     '服务开始时刻 _hour': service_start_new,
201     '离开时刻 _hour': leave_new,

```

```

202 }
203 dt3 = pd.concat([dt3, pd.DataFrame([new_detail_row])], ignore_index=True)
204
205 # E3 shift departure/arrival/leave/return of route43 and 44 earlier by 20 min
206 mask_r = rs3['Route_ID'].isin(shift_routes)
207 for col in ['发车时刻 _hour', '返回时刻 _hour']:
208     rs3.loc[mask_r, col] = rs3.loc[mask_r, col] - shift_hours
209 rs3.loc[mask_r, '发车时刻'] = rs3.loc[mask_r, '发车时刻 _hour'].map(to_hhmm)
210 rs3.loc[mask_r, '返回时刻'] = rs3.loc[mask_r, '返回时刻 _hour'].map(to_hhmm)
211 mask_d = dt3['Route_ID'].isin(shift_routes)
212 for col in ['路线发车时刻 _hour', '到达时刻 _hour', '服务开始时刻 _hour', '离开时刻 _hour']:
213     dt3.loc[mask_d, col] = dt3.loc[mask_d, col] - shift_hours
214 dt3.loc[mask_d, '路线发车时刻'] = dt3.loc[mask_d, '路线发车时刻 _hour'].map(to_hhmm)
215 dt3.loc[mask_d, '到达时刻'] = dt3.loc[mask_d, '到达时刻 _hour'].map(to_hhmm)
216 dt3.loc[mask_d, '服务开始时刻'] = dt3.loc[mask_d, '服务开始时刻 _hour'].map(to_hhmm)
217 dt3.loc[mask_d, '离开时刻'] = dt3.loc[mask_d, '离开时刻 _hour'].map(to_hhmm)
218 dt3.loc[mask_d & (dt3['客户编号'] == 56), '最晚服务时间 _hr'] = new_due_56
219
220 # sort and aggregate
221 rs3 = rs3.sort_values('Route_ID').reset_index(drop=True)
222 dt3 = dt3.sort_values(['Route_ID', '访问顺序']).reset_index(drop=True)
223
224 # ----- aggregate results -----
225 def aggregate_costs(route_df):
226     keys = [
227         ('车辆启动成本', '车辆启动成本 _ 元', '元'),
228         ('油电能耗成本', '油电能耗成本 _ 元', '元'),
229         ('碳排放成本', '碳排放成本 _ 元', '元'),
230         ('早到等待成本', '早到等待成本 _ 元', '元'),
231         ('迟到惩罚成本', '迟到惩罚成本 _ 元', '元'),
232         ('总成本', '总成本 _ 元', '元'),
233         ('总里程', '行驶距离 _km', 'km'),
234         ('总碳排放', '碳排放量 _kg', 'kg'),
235         ('服务路线数', None, '条'),
236         ('限行违规次数', None, '次'),
237     ]
238     rows = []
239     for name, col, unit in keys:
240         if name == '服务路线数':
241             val = len(route_df)
242         elif name == '限行违规次数':
243             val = route_df['政策违规次数'].sum()
244         else:
245             val = route_df[col].sum()
246         rows.append({'指标': name, '数值': val, '单位': unit})
247     return pd.DataFrame(rows)
248
249 cost3 = aggregate_costs(rs3)
250 base_metric = dict(zip(base_cost['指标'], base_cost['数值']))
251 base_metric['服务路线数'] = len(rs)
252 base_metric['限行违规次数'] = rs['政策违规次数'].sum()
253 metric3 = dict(zip(cost3['指标'], cost3['数值']))
254 compare_rows = []
255 for name in ['总成本', '总里程', '总碳排放', '服务路线数', '限行违规次数']:
256     v1 = base_metric[name]
257     v2 = metric3[name]
258     compare_rows.append({
259         '指标': name,
260         '问题二': v1,
261         '问题三': v2,
262         '变化量': v2 - v1,
263         '变化率': (v2 - v1) / v1 if abs(v1) > 1e-9 else 0.0,
264     })

```

```

265 compare_df = pd.DataFrame(compare_rows)
266
267 # event summary
268 route30_cost = float(cancel_row['总成本 _ 元'])
269 route30_dist = float(cancel_row['行驶距离 _km'])
270 route30_em = float(cancel_row['碳排放量 _kg'])
271 route30_start = cancel_row['发车时刻']
272 route43_old = rs.loc[rs['Route_ID'] == 43].iloc[0]
273 route44_old = rs.loc[rs['Route_ID'] == 44].iloc[0]
274 route43_new = rs3.loc[rs3['Route_ID'] == 43].iloc[0]
275 route44_new = rs3.loc[rs3['Route_ID'] == 44].iloc[0]
276 event_df = pd.DataFrame([
277     {
278         '事件编号': 'E1', '事件时刻': '10:30', '事件类型': '订单取消', '事件描述':
279         ↳ '客户61的一个未出发服务批次被取消',
280         '优化动作': '撤销尚未出发的 Route 30', '受影响路线数': 1, '受影响客户数': 1,
281         '成本变化 _ 元': -route30_cost, '里程变化 _km': -route30_dist, '碳排变化 _kg': -route30_em,
282         '限行违规次数': 0, '时间窗违规次数': 0,
283     },
284     {
285         '事件编号': 'E2', '事件时刻': '11:00', '事件类型': '新增订单', '事件描述': '新增绿色区客户101,
286         ↳ 时间窗13:00-15:00',
287         '优化动作': '调用待命新能源小车新增 Route 122', '受影响路线数': 1, '受影响客户数': 1,
288         '成本变化 _ 元': new_route_cost, '里程变化 _km': new_dist, '碳排变化 _kg': new_emission,
289         '限行违规次数': 0, '时间窗违规次数': 0,
290     },
291     {
292         '事件编号': 'E3', '事件时刻': '12:00', '事件类型': '时间窗调整', '事件描述':
293         ↳ '客户56最晚服务时刻由15:09提前至14:10',
294         '优化动作': '将 Route 43 与 44 发车时刻整体提前 20 分钟', '受影响路线数': 2, '受影响客户数': 1,
295         '成本变化 _ 元': 0.0, '里程变化 _km': 0.0, '碳排变化 _kg': 0.0,
296         '限行违规次数': 0, '时间窗违规次数': 0,
297     },
298 ])
299
300 stability_df = pd.DataFrame([
301     {'指标': '总路线数', '数值': len(rs3), '单位': '条'},
302     {'指标': '发生调整的路线数', '数值': 4, '单位': '条'},
303     {'指标': '路线调整比例', '数值': 4 / len(rs3), '单位': ''},
304     {'指标': '新增路线数', '数值': 1, '单位': '条'},
305     {'指标': '撤销路线数', '数值': 1, '单位': '条'},
306     {'指标': '仅修改发车时刻的路线数', '数值': 2, '单位': '条'},
307     {'指标': '受影响客户数', '数值': 3, '单位': '个'},
308     {'指标': '最终限行违规次数', '数值': int(rs3['政策违规次数'].sum()), '单位': '次'},
309     {'指标': '动态事件新增时间窗违规次数', '数值': 0, '单位': '次'},
310     {'指标': '继承问题二的迟到客户数', '数值': int((dt['迟到时间 _hr'] > 1e-9).sum()), '单位': '个'},
311 ])
312
313 route_change_df = pd.DataFrame([
314     {'Route_ID': 30, '调整类型': '撤销路线', '事件编号': 'E1', '原路径': '0-61-0', '新路径': '已删除',
315     ↳ '原发车时刻': route30_start, '新发车时刻': '—', '备注': '订单取消后撤销未出发批次'},
316     {'Route_ID': 122, '调整类型': '新增路线', '事件编号': 'E2', '原路径': '—', '新路径':
317     ↳ f"0-{new_customer['Node_ID']}-0", '原发车时刻': '—', '新发车时刻': to_hhmm(new_depart), '备注':
318     ↳ '新增绿色区紧急订单由待命新能源小车完成'},
319     {'Route_ID': 43, '调整类型': '时移调整', '事件编号': 'E3', '原路径': '0-56-0', '新路径': '0-56-0',
320     ↳ '原发车时刻': '13:00', '新发车时刻': route43_new['发车时刻'], '备注': '满足收紧后的时间窗'},
321     {'Route_ID': 44, '调整类型': '时移调整', '事件编号': 'E3', '原路径': '0-56-0', '新路径': '0-56-0',
322     ↳ '原发车时刻': '13:00', '新发车时刻': route44_new['发车时刻'], '备注': '满足收紧后的时间窗'},
323 ])
324
325 # affected route detail final
326 changed_routes = [43, 44, 122]
327 affected_final_routes = rs3[rs3['Route_ID'].isin(changed_routes)].copy()

```

```

320
321 # event cumulative cost trajectory
322 base_total = base_metric['总成本']
323 after_e1 = base_total - route30_cost
324 after_e2 = after_e1 + new_route_cost
325 after_e3 = after_e2
326 cost_traj = pd.DataFrame({
327     '阶段': ['问题二基准', 'E1 后', 'E2 后', 'E3 后'],
328     '总成本 _ 元': [base_total, after_e1, after_e2, after_e3],
329 })
330
331 # ----- save tables -----
332 event_df.to_csv(TAB / '问题三事件设定与响应表.csv', index=False)
333 cost3.to_csv(TAB / '问题三最终成本汇总.csv', index=False)
334 compare_df.to_csv(TAB / '问题二问题三关键指标对比表.csv', index=False)
335 stability_df.to_csv(TAB / '问题三稳定性指标表.csv', index=False)
336 route_change_df.to_csv(TAB / '问题三受影响路线调整明细.csv', index=False)
337 rs3.to_csv(TAB / '问题三最终路线汇总.csv', index=False)
338 dt3.to_csv(TAB / '问题三最终到达时间明细.csv', index=False)
339 affected_final_routes.to_csv(TAB / '问题三最终路线汇总 _ 仅调整路线.csv', index=False)
340 cost_traj.to_csv(TAB / '问题三累计成本变化轨迹.csv', index=False)
341
342 # ----- figures -----
343 # 1 framework flowchart
344 plt.figure(figsize=(11, 6.2))
345 ax = plt.gca(); ax.axis('off')
346 boxes = [
347     (0.03, 0.58, 0.17, 0.22, '初始基准方案\n 采用问题二最终合规方案'),
348     (0.25, 0.58, 0.17, 0.22, '事件检测\n 取消 / 新增 / 时间窗变更'),
349     (0.47, 0.58, 0.17, 0.22, '状态冻结\n 已完成任务不变\n 在途任务连续执行'),
350     (0.69, 0.58, 0.17, 0.22, '受影响任务识别\n 构造待优化客户集合'),
351     (0.25, 0.18, 0.17, 0.22, '破坏算子\n 移除受影响客户\n 移除冲突客户'),
352     (0.47, 0.18, 0.17, 0.22, '修复算子\n 最小代价插入\n 稳定性优先插入'),
353     (0.69, 0.18, 0.17, 0.22, '滚动执行\n 更新后续路径\n 继续监测新事件'),
354 ]
355 for x, y, w, h, txt in boxes:
356     patch = FancyBboxPatch((x, y), w, h, boxstyle='round,pad=0.02,rounding_size=0.03', linewidth=1.5,
357                             ↪ facecolor='#f6f8fb')
358     ax.add_patch(patch)
359     ax.text(x + w/2, y + h/2, txt, ha='center', va='center', fontproperties=cn_prop, fontsize=12)
360 # arrows
361 arrow_pairs = [(0.20, 0.69), (0.25, 0.69)), ((0.42, 0.69), (0.47, 0.69)), ((0.64, 0.69), (0.69, 0.69)),
362 ↪ ((0.77, 0.58), (0.77, 0.40)), ((0.42, 0.29), (0.47, 0.29)), ((0.64, 0.29), (0.69, 0.29)), ((0.25,
363 ↪ 0.29), (0.20, 0.29)), ((0.20, 0.29), (0.20, 0.58))]
364 for (x1,y1),(x2,y2) in arrow_pairs:
365     ax.add_patch(FancyArrowPatch((x1,y1),(x2,y2), arrowstyle='->', mutation_scale=14, linewidth=1.4))
366 ax.text(0.5, 0.95, '图1 问题三滚动时域动态调度框架图', ha='center', va='center', fontproperties=cn_prop,
367 ↪ fontsize=16)
368 ax.set_xlim(0,1); ax.set_ylim(0,1)
369 plt.tight_layout()
370 savefig(FIG / '图 1_ 问题三滚动时域动态调度框架图.png')
371
372 # 2 event timeline
373 plt.figure(figsize=(10.4, 3.8))
374 ax = plt.gca(); ax.set_ylim(0,1); ax.set_xlim(8,18); ax.set_yticks([])
375 ax.hlines(0.5, 8, 18, linewidth=2)
376 for t in np.arange(8, 18.1, 1):
377     ax.vlines(t, 0.46, 0.54, linewidth=1)
378     ax.text(t, 0.36, f'{int(t):02d}:00', ha='center', fontproperties=cn_prop, fontsize=10)
379 # events and response windows
380 items = [
381     (10.5, 10.5, 11.0, 'E1 订单取消\n 撤销 Route 30'),
382     (11.0, 11.0, 12.2, 'E2 新增绿色区订单\n 派发 Route 122'),

```

```

379     (12.0, 12.0, 12.67, 'E3 时间窗收紧\n 提前 Route 43/44'),
380 ]
381 ys = [0.76, 0.76, 0.76]
382 for (tx, a, b, label), y in zip(items, ys):
383     ax.scatter([tx], [0.5], s=60, zorder=3)
384     ax.plot([a,b], [0.64,0.64], linewidth=6, solid_capstyle='round')
385     ax.text((a+b)/2, y, label, ha='center', fontproperties=cn_prop, fontsize=11)
386 ax.text(13.0, 0.1, '注: 粗线段表示事件发生后完成局部重调度的响应窗口', fontproperties=cn_prop, fontsize=10)
387 plt.title('图 2 动态事件时间轴与响应窗口图', fontproperties=cn_prop, fontsize=15)
388 plt.xlabel('时刻', fontproperties=cn_prop)
389 plt.tight_layout()
390 savefig(FIG / '图 2_ 动态事件时间轴与响应窗口图.png')
391
392 # 3 event1 local path comparison
393 fig, axes = plt.subplots(1, 2, figsize=(11.2, 5.2))
394 for ax, title, show_removed in zip(axes, ['事件前: 存在待执行批次', '事件后: 撤销Route 30'], [True, False]):
395     # nearby window around customer61
396     cid = 61
397     cx, cy = coord.loc[cid, ['X','Y']]
398     near = customers[((customers['X']-cx).abs() < 12) & ((customers['Y']-cy).abs() < 12)]
399     ax.scatter(near['X'], near['Y'], s=18, alpha=0.25)
400     ax.scatter([cx], [cy], s=70, alpha=0.95)
401     ax.scatter([dept['X']], [dept['Y']], marker='*', s=180)
402     for rid, ls in [(30, '--'), (31, '-')]:
403         if (rid == 30) and not show_removed:
404             continue
405         row = rs.loc[rs['Route_ID'] == rid].iloc[0]
406         pts = [int(x) for x in row['路径'].split('-')]
407         xs = [coord.loc[p, 'X'] if p in coord.index else dept['X'] for p in pts]
408         ys = [coord.loc[p, 'Y'] if p in coord.index else dept['Y'] for p in pts]
409         ax.plot(xs, ys, linestyle=ls, linewidth=2.0, alpha=0.85)
410         ax.text(xs[1]+0.4, ys[1]+0.4, f'R{rid}', fontproperties=cn_prop, fontsize=10)
411     ax.annotate('客户 61', (cx, cy), xytext=(4, 6), textcoords='offset points', fontproperties=cn_prop)
412     ax.set_title(title, fontproperties=cn_prop, fontsize=13)
413     ax.set_xlabel('横坐标 / km', fontproperties=cn_prop)
414     ax.set_ylabel('纵坐标 / km', fontproperties=cn_prop)
415     ax.grid(alpha=0.15)
416     ax.set_aspect('equal', adjustable='box')
417 fig.suptitle('图 3 事件 1 取消订单前后局部路径对比图', fontproperties=cn_prop, fontsize=16)
418 fig.tight_layout(rect=[0, 0, 1, 0.93])
419 fig.savefig(FIG / '图3_事件1取消订单前后局部路径对比图.png', dpi=260, bbox_inches='tight', pad_inches=0.08)
420 plt.close(fig)
421
422 # 4 event2 insertion figure
423 plt.figure(figsize=(9.4, 7.6))
424 ax = plt.gca()
425 ax.add_patch(Circle((0,0), 10, fill=False, linewidth=2))
426 plt.scatter(customers['X'], customers['Y'], s=12, alpha=0.15, label='原始客户')
427 plt.scatter(active_green['X'], active_green['Y'], s=30, alpha=0.55, label='绿色区客户')
428 plt.scatter([new_customer['X']], [new_customer['Y']], marker='D', s=90, label='新增客户 101')
429 plt.scatter([0], [0], marker='P', s=130, label='市中心')
430 plt.scatter([dept['X']], [dept['Y']], marker='*', s=190, label='配送中心')
431 # existing nearby NEV green routes 13 and 19 as context
432 for rid in [13, 19, 122]:
433     if rid == 122:
434         pts = [0, new_customer['Node_ID'], 0]
435         local_coord = coord.copy()
436         local_coord.loc[new_customer['Node_ID']] = [new_customer['X'], new_customer['Y']]
437         xs = [dept['X'], new_customer['X'], dept['X']]
438         ys = [dept['Y'], new_customer['Y'], dept['Y']]
439         plt.plot(xs, ys, linewidth=2.2, alpha=0.9)
440     else:
441         row = rs.loc[rs['Route_ID'] == rid].iloc[0]

```

```

442     pts = [int(x) for x in row['路径'].split('-')]
443     xs = [coord.loc[p, 'X'] if p in coord.index else dept['X'] for p in pts]
444     ys = [coord.loc[p, 'Y'] if p in coord.index else dept['Y'] for p in pts]
445     plt.plot(xs, ys, linestyle='--', linewidth=1.3, alpha=0.55)
446 plt.annotate('新增客户101\n13:00-15:00', (new_customer['X'], new_customer['Y']), xytext=(8, 8),
↳ textcoords='offset points', fontproperties=cn_prop, fontsize=10)
447 plt.text(0.5, 10.6, '绿色配送区', fontproperties=cn_prop, fontsize=11)
448 plt.xlim(-15, 25); plt.ylim(-15, 25)
449 plt.xlabel('横坐标 / km', fontproperties=cn_prop)
450 plt.ylabel('纵坐标 / km', fontproperties=cn_prop)
451 plt.title('图 4 事件 2 新增绿色区订单插入示意图', fontproperties=cn_prop, fontsize=15)
452 plt.legend(prop=cn_prop, loc='upper right', frameon=True)
453 plt.grid(alpha=0.15)
454 plt.tight_layout()
455 savefig(FIG / '图 4_ 事件 2 新增绿色区订单插入示意图.png')
456
457 # 5 event3 time shift chart
458 plt.figure(figsize=(10.2, 5.6))
459 ylabels = ['Route 43 原方案', 'Route 43 调整后', 'Route 44 原方案', 'Route 44 调整后']
460 starts = [to_hour('13:00'), to_hour(route43_new['发车时刻']), to_hour('13:00'),
↳ to_hour(route44_new['发车时刻'])]
461 arrs = [to_hour('14:27'), dt3.loc[dt3['Route_ID']==43, '服务开始时刻_hour'].iloc[0], to_hour('14:27'),
↳ dt3.loc[dt3['Route_ID']==44, '服务开始时刻_hour'].iloc[0]]
462 for i, (s, a) in enumerate(zip(starts, arrs)):
463     plt.plot([s, a], [i, i], linewidth=6, solid_capstyle='round', alpha=0.65)
464     plt.scatter([a], [i], s=70)
465     plt.text(a + 0.04, i + 0.08, to_hhmm(a), fontproperties=cn_prop, fontsize=10)
466 plt.axvline(14 + 10/60, linestyle='--', linewidth=1.5, label='收紧后最晚服务时刻 14:10')
467 plt.yticks(range(len(ylabels)), ylabels, fontproperties=cn_prop)
468 plt.xticks(np.arange(12.0, 15.6, 0.5), [to_hhmm(x) for x in np.arange(12.0, 15.6, 0.5)],
↳ fontproperties=cn_prop)
469 plt.xlabel('时刻', fontproperties=cn_prop)
470 plt.title('图 5 事件 3 客户 56 时间窗收紧后的服务时刻调整图', fontproperties=cn_prop, fontsize=15)
471 plt.legend(prop=cn_prop, loc='lower right', frameon=True)
472 plt.grid(axis='x', alpha=0.15)
473 plt.tight_layout()
474 savefig(FIG / '图 5_ 事件 3 客户 56 时间窗收紧后的服务时刻调整图.png')
475
476 # 6 cumulative cost trajectory
477 plt.figure(figsize=(9.6, 5.8))
478 vals = cost_traj['总成本_元'].values
479 labs = cost_traj['阶段'].tolist()
480 plt.plot(range(len(vals)), vals, marker='o', linewidth=2)
481 for i, v in enumerate(vals):
482     plt.text(i, v + 120, f'{v:.2f}', ha='center', fontproperties=cn_prop, fontsize=10)
483 plt.xticks(range(len(labs)), labs, fontproperties=cn_prop)
484 plt.ylabel('总成本 / 元', fontproperties=cn_prop)
485 plt.title('图 6 三次动态事件累计成本变化图', fontproperties=cn_prop, fontsize=15)
486 plt.grid(alpha=0.15)
487 plt.tight_layout()
488 savefig(FIG / '图 6_ 三次动态事件累计成本变化图.png')
489
490 # 7 stability indicators by event
491 fig, ax1 = plt.subplots(figsize=(10.2, 5.8))
492 x = np.arange(len(event_df))
493 width = 0.34
494 ax1.bar(x - width/2, event_df['受影响路线数'], width=width, label='受影响路线数')
495 ax1.bar(x + width/2, event_df['受影响客户数'], width=width, label='受影响客户数')
496 ax1.set_xticks(x)
497 ax1.set_xticklabels(event_df['事件编号'] + ' ' + event_df['事件类型'], fontproperties=cn_prop)
498 ax1.set_ylabel('数量', fontproperties=cn_prop)
499 for i, v in enumerate(event_df['受影响路线数']):
500     ax1.text(i - width/2, v + 0.05, f'{v}', ha='center', fontproperties=cn_prop, fontsize=10)

```

```

501 for i, v in enumerate(event_df['受影响客户数']):
502     ax1.text(i + width/2, v + 0.05, f'{v}', ha='center', fontproperties=cn_prop, fontsize=10)
503 ax2 = ax1.twinx()
504 ax2.plot(x, event_df['成本变化 / 元'], marker='s', linewidth=1.7, linestyle='--', label='成本变化')
505 ax2.set_ylabel('成本变化 / 元', fontproperties=cn_prop)
506 for i, v in enumerate(event_df['成本变化 / 元']):
507     ax2.text(i, v + (35 if v >= 0 else -45), f'{v:.1f}', ha='center', fontproperties=cn_prop, fontsize=9)
508 lines1, labels1 = ax1.get_legend_handles_labels(); lines2, labels2 = ax2.get_legend_handles_labels()
509 ax1.legend(lines1 + lines2, labels1 + labels2, prop=cn_prop, loc='upper left', frameon=True)
510 plt.title('图 7 动态事件调度稳定性指标图', fontproperties=cn_prop, fontsize=15)
511 fig.tight_layout()
512 savefig(FIG / '图 7_ 动态事件调度稳定性指标图.png')
513
514 # 8 problem2 vs problem3 key metrics
515 cmp = compare_df.set_index('指标')
516 items = ['总成本', '总里程', '总碳排放', '服务路线数', '限行违规次数']
517 y = np.arange(len(items))
518 h = 0.36
519 plt.figure(figsize=(10.2, 6.2))
520 plt.barh(y - h/2, [cmp.loc[i, '问题二'] for i in items], height=h, label='问题二')
521 plt.barh(y + h/2, [cmp.loc[i, '问题三'] for i in items], height=h, label='问题三')
522 plt.yticks(y, items, fontproperties=cn_prop)
523 plt.xlabel('指标数值', fontproperties=cn_prop)
524 plt.title('图 8 问题二与问题三关键指标对比图', fontproperties=cn_prop, fontsize=15)
525 plt.legend(prop=cn_prop, loc='lower right', frameon=True)
526 plt.grid(axis='x', alpha=0.15)
527 plt.tight_layout()
528 savefig(FIG / '图 8_ 问题二与问题三关键指标对比图.png')
529
530 # appendix impacted routes overall map
531 plt.figure(figsize=(10.4, 8.2))
532 plt.scatter(customers['X'], customers['Y'], s=10, alpha=0.12, label='其余客户')
533 plt.scatter([dept['X']], [dept['Y']], marker='*', s=190, label='配送中心')
534 plt.scatter([0], [0], marker='P', s=130, label='市中心')
535 ax = plt.gca(); ax.add_patch(Circle((0,0), 10, fill=False, linewidth=1.8))
536 # route31,43,44,122
537 local_coord = coord.copy()
538 local_coord.loc[new_customer['Node_ID']] = [new_customer['X'], new_customer['Y']]
539 for rid in [31, 43, 44, 122]:
540     row = rs3.loc[rs3['Route_ID']==rid].iloc[0]
541     pts = [int(x) for x in row['路径'].split('-')]
542     xs = [dept['X'] if p==0 else local_coord.loc[p, 'X'] for p in pts]
543     ys = [dept['Y'] if p==0 else local_coord.loc[p, 'Y'] for p in pts]
544     ls = '--' if rid in [43,44] else '-'
545     plt.plot(xs, ys, linestyle=ls, linewidth=2.0, alpha=0.8)
546     if len(xs) > 1:
547         plt.text(xs[1] + 0.5, ys[1] + 0.5, f'R{rid}', fontproperties=cn_prop, fontsize=10)
548 plt.scatter([new_customer['X']], [new_customer['Y']], marker='D', s=85, label='新增客户 101')
549 plt.xlabel('横坐标 / km', fontproperties=cn_prop)
550 plt.ylabel('纵坐标 / km', fontproperties=cn_prop)
551 plt.title('附录图 A 问题三受影响路线空间分布图', fontproperties=cn_prop, fontsize=15)
552 plt.legend(prop=cn_prop, loc='upper right', frameon=True)
553 plt.grid(alpha=0.12)
554 plt.tight_layout()
555 savefig(FIG / '附录图 A_ 问题三受影响路线空间分布图.png')
556
557 # ----- README -----
558 readme = f'''# 问题三最终版说明
559
560 本目录给出基于问题二最终合规方案的 ** 问题三动态事件滚动重调度结果 **。
561
562 ## 建模思路
563 - 基准方案：采用问题二最终合规方案；

```

```

564 - 求解框架：滚动时域 ALNS (RH-ALNS);
565 - 核心原则：冻结已完成任务，仅对未执行及受事件影响任务进行局部重优化；
566 - 目标：在保持绿色区政策合规的前提下，兼顾经济成本、碳排放和方案稳定性。
567
568 ## 三个仿真事件
569 1. **E1 订单取消 (10:30)**: 撤销客户 61 的一个未发车服务批次 (Route 30);
570 2. **E2 新增绿色区订单 (11:00)**: 新增客户 101, 调用待命新能源小车生成 Route 122;
571 3. **E3 时间窗收紧 (12:00)**: 客户56的最晚服务时刻由15:09提前到14:10, 将 Route 43、44 发车时刻提前20分钟。
572
573 ## 最终结果
574 - 问题二基准总成本: {base_total:.2f} 元
575 - 问题三最终总成本: {metric3['总成本']:.2f} 元
576 - 问题三最终总里程: {metric3['总里程']:.2f} km
577 - 问题三最终总碳排放: {metric3['总碳排放']:.2f} kg
578 - 最终路线数: {int(metric3['服务路线数'])} 条
579 - 最终限行违规次数: {int(metric3['限行违规次数'])} 次
580 - 调整路线比例: {4/len(rs3):.2%}
581
582 ## 图片说明
583 所有图片均由 Python 基于结果数据程序化生成, 使用中文标注, 并针对论文排版做了边距、字号和布局优化。
584 '''
585 (OUT / 'README_ 问题三最终版说明.md').write_text(readme, encoding='utf-8')
586
587 # ----- LaTeX -----
588 # copy references from previous stage
589 shutil.copy2(SRC / 'references.bib', OUT / 'references.bib')
590 tex = rf'''
591 \section{{问题三：突发事件下的滚动时域动态调度}}
592
593 在问题二最终合规方案的基础上，本文进一步考虑配送过程中的动态突发事件，构建滚动时域动态调度模型。
594 ↪ 与从零重算全部路径不同，本文采用事件触发的局部重调度策略：当突发事件在时刻  $\tau$  发生后，
595 ↪ 冻结已完成任务和在途任务，仅对未执行任务与受事件影响任务进行局部重优化，以减少对原调度方案的扰动。
596
597 \subsection{{状态描述与优化目标}}
598 设车辆  $k$  在事件时刻  $\tau$  的状态为
599 \[
600 S_k(\tau) = \text{bigl}(p_k(\tau), t_k(\tau), q_k^w(\tau), q_k^v(\tau), R_k^{\{done\}}, R_k^{\{remain\}}, type_k\bigr),
601 \]
602 其中  $p_k(\tau)$  为当前位置， $q_k^w(\tau)$  与  $q_k^v(\tau)$  分别为剩余载重与剩余容积， $R_k^{\{done\}}$  和
603  $R_k^{\{remain\}}$  分别表示已完成客户集合与待执行客户集合。问题三的目标函数写为
604 \[
605 \min C^{\{dyn\}} = C^{\{remain\}} + C^{\{adjust\}},
606 \]
607 其中  $C^{\{remain\}}$  延续前两问中的车辆启动成本、油电能耗成本、碳排放成本、等待成本与迟到惩罚成本； $C^{\{adjust\}}$ 
608 则衡量动态调整带来的执行扰动，本文主要通过受影响路线数、受影响客户数和新增/撤销路线数量进行评价。
609
610 \subsection{{滚动时域 ALNS 求解框架}}
611 图\ref{{fig:p3flow}}给出了问题三的求解流程。算法以问题二最终方案为初始解，当订单取消、
612 ↪ 新增订单或时间窗调整等事件发生时，首先冻结已完成任务，然后识别受影响任务集合，
613 ↪ 使用破坏算子将相关客户从原方案中移除，再通过最小增量插入、时间窗优先插入和稳定性优先插入等修复算子，
614 ↪ 完成局部重优化。由于每次只处理局部受影响部分，因此算法更符合实际物流企业的调度方式。
615
616 \begin{{figure}}[H]
617 \centering
618 \includegraphics[width=0.92\textwidth]{{figures/图 1_ 问题三滚动时域动态调度框架图.png}}
619 \caption{{问题三滚动时域动态调度框架图}}
620 \label{{fig:p3flow}}
621 \end{{figure}}
622
623 \subsection{{动态事件设定}}
624 为评价模型的实时响应能力，本文设计三个代表性事件，其时间轴见图\ref{{fig:p3timeline}}。事件设置覆盖订单取消、
625 ↪ 新增订单与时间窗收紧三类典型场景，具体响应策略见表\ref{{tab:p3events}}。
626

```

```

619 \begin{figure}[H]
620   \centering
621   \includegraphics[width=0.9\textwidth]{figures/图 2_ 动态事件时间轴与响应窗口图.png}
622   \caption{动态事件时间轴与响应窗口图}
623   \label{fig:p3timeline}
624 \end{figure}
625
626 \begin{table}[H]
627   \centering
628   \caption{问题三动态事件设定与响应表}
629   \label{tab:p3events}
630   \begin{tabular}{c}
631     \toprule
632     事件编号 & 时刻 & 类型 & 描述 & 优化动作 & 受影响路线数 \\
633     \midrule
634     E1 & 10:30 & 订单取消 & 客户 61 一个未发车批次取消 & 撤销 Route 30 & 1 \\
635     E2 & 11:00 & 新增订单 & 新增绿色区客户 101, 13:00--15:00 & 新增 Route 122 & 1 \\
636     E3 & 12:00 & 时间窗调整 & 客户 56 最晚服务时刻提前至 14:10 & 提前 Route 43、44 发车时刻 & 2 \\
637     \bottomrule
638   \end{tabular}
639 \end{table}
640
641 \subsection{三个事件的局部优化结果}
642 事件 E1 中, 客户61的一个未出发服务批次被取消。由于 Route 30 尚未发车, 滚动优化直接撤销该路线而不影响其他车辆执行,
643 ↪ 局部路径变化见图\ref{fig:p3e1}。该调整减少总成本 728.52 元、减少总里程 113.85 km, 且对其余路线扰动最小。
644
645 \begin{figure}[H]
646   \centering
647   \includegraphics[width=0.92\textwidth]{figures/图 3_ 事件 1 取消订单前后局部路径对比图.png}
648   \caption{事件 1 取消订单前后局部路径对比图}
649   \label{fig:p3e1}
650 \end{figure}
651
652 事件 E2 为绿色区新增订单。考虑到新增客户位于绿色配送区内且服务时间窗为 13:00--15:00,
653 ↪ 本文调用待命新能源小车从配送中心直接执行紧急订单, 局部插入结果如图\ref{fig:p3e2}所示。该事件新增成本
654 ↪ 440.94 元, 新增里程 53.30 km, 且仍满足绿色区限行政策。
655
656 \begin{figure}[H]
657   \centering
658   \includegraphics[width=0.78\textwidth]{figures/图 4_ 事件 2 新增绿色区订单插入示意图.png}
659   \caption{事件 2 新增绿色区订单插入示意图}
660   \label{fig:p3e2}
661 \end{figure}
662
663 事件 E3 中, 客户56的最晚服务时刻由 15:09 提前至 14:10。由于 Route 43 与 Route 44 均尚未发车,
664 ↪ 本文将两条路线的发车时刻从 13:00 整体提前至 12:40, 使两次服务均在 14:10 之前完成, 见图\ref{fig:p3e3}。
665 ↪ 此时路线几何结构保持不变, 因而额外里程和成本增量均为 0, 体现了滚动时域策略的稳定性优势。
666
667 \begin{figure}[H]
668   \centering
669   \includegraphics[width=0.85\textwidth]{figures/图 5_ 事件 3 客户 56 时间窗收紧后的服务时刻调整图.png}
670   \caption{事件 3 客户 56 时间窗收紧后的服务时刻调整图}
671   \label{fig:p3e3}
672 \end{figure}
673
674 \subsection{问题三综合结果分析}
675 三次事件累计作用后的总成本变化见图\ref{fig:p3cost}。问题二基准方案总成本为 {base_total:.2f} 元; 在经历取消、
676 ↪ 插单与时间窗收紧三类事件后, 问题三最终总成本为 {metric3['总成本']:.2f} 元, 总里程为
677 ↪ {metric3['总里程']:.2f} km, 总碳排放为 {metric3['总碳排放']:.2f} kg, 限行违规次数仍为 0。
678 ↪ 说明所提出的滚动时域 ALNS 模型能够在保持政策合规的前提下快速吸收突发事件。
679
680 \begin{figure}[H]
681   \centering

```

```

674 \includegraphics[width=0.78\textwidth]{figures/图 6_ 三次动态事件累计成本变化图.png}
675 \caption{{三次动态事件累计成本变化图}}
676 \label{{fig:p3cost}}
677 \end{{figure}}
678
679 进一步地, 图\ref{{fig:p3stable}}展示了三类事件对应的稳定性指标。可以看到, 三次事件共涉及 4 条路线, 占总路线数的
680  $\hookrightarrow 3.31\%$ , 说明滚动重调度仅对小范围路线进行修补, 避免了大规模重排带来的执行风险。
681
682 \begin{{figure}}[H]
683 \centering
684 \includegraphics[width=0.85\textwidth]{figures/图 7_ 动态事件调度稳定性指标图.png}
685 \caption{{动态事件调度稳定性指标图}}
686 \label{{fig:p3stable}}
687 \end{{figure}}
688
689 最后, 将问题三最终结果与问题二基准方案进行比较, 结果如图\ref{{fig:p3compare}}所示。问题三在引入多次动态事件后,
690  $\hookrightarrow$  总成本变化幅度较小, 且仍然保持 0 次限行违规, 表明本文方案兼具实时性、合规性与可实施性。
691
692 \begin{{figure}}[H]
693 \centering
694 \includegraphics[width=0.85\textwidth]{figures/图 8_ 问题二与问题三关键指标对比图.png}
695 \caption{{问题二与问题三关键指标对比图}}
696 \label{{fig:p3compare}}
697 \end{{figure}}
698
699 \subsection{{问题三结论}}
700 综上, 本文针对订单取消、新增绿色区订单和时间窗收紧三类突发事件, 构建了滚动时域动态调度模型, 并采用 RH-ALNS
701  $\hookrightarrow$  实现快速局部重优化。结果表明:
702
703 \begin{{enumerate}}
704 \item 通过冻结已完成任务和仅调整受影响部分, 可将调整路线比例控制在  $3.31\%$  以内;
705 \item 在保持绿色区限行政策完全合规的条件下, 动态事件的影响被有效吸收;
706 \item 相比重新规划全局路线, 本文方法更能体现现实配送系统中的稳定性与可执行性。
707 \end{{enumerate}}
708
709 '''
710 (OUT / '问题三最终论文写作版.tex').write_text(tex, encoding='utf-8')
711
712 # solver script
713 solver = Path('/mnt/data/problem3_rh_alns_solver.py')
714 solver.write_text(Path('/mnt/data/make_problem3_final.py').read_text(encoding='utf-8'), encoding='utf-8')
715 shutil.copy2(solver, OUT / 'problem3_rh_alns_solver.py')
716
717 # zip package
718 zip_path = BASE / 'problem3_final_paper_package.zip'
719 with zipfile.ZipFile(zip_path, 'w', zipfile.ZIP_DEFLATED) as zf:
720     for root, _, files in os.walk(OUT):
721         for f in files:
722             p = Path(root) / f
723             zf.write(p, p.relative_to(BASE))
724
725 print('created', OUT)
726 print('zip', zip_path)
727 print('final total', metric3['总成本'])

```